



Conservatoire National des Arts et Métiers
Centre Paris

MÉMOIRE DE MASTER

présenté par : **Ahmed Atmane**

pour obtenir le grade de : **Master en Informatique**

Spécialité : **Réseaux & Objets Connectés (MR11606C)**

soutenance prévue : **septembre 2026**

InferRouter-LLM

Routage dynamique d'intents réseau vers des LLM spécialisés
par estimation sémantique de complexité
et évaluation automatique de qualité

Mémoire dirigé par :

[Tuteur de mémoire], CNAM Paris

Jury

[À compléter]

[À compléter]

[À compléter]

CNAM Paris

Titre, établissement

Titre, établissement

Tuteur

Président

Examineur

Travail individuel — Continuité du projet RSX218

Acknowledgements

Insert here the text of your acknowledgements

Abstract

Insert here your abstract in English.

Keywords: keyword 1, keyword 2, ... keyword 10 (maximum).

Résumé

Insérer ici votre résumé en français suivi des mots-clés.

Mots-clés : mot-clé 1, mot-clé 2 , ... , mot-clé 10 (maximum).

Contents

Acknowledgements	i
Abstract	ii
Résumé	iii
List of Tables	1
 Chapters	
1 Introduction	2
1.1 Contexte et motivation	2
1.2 Problématique	3
1.3 Contributions	4
1.3.0.0.1 Estimateur sémantique de complexité d'intents réseau. . . .	4
1.3.0.0.2 Routage tri-critère sous contraintes dures par intent.	5
1.3.0.0.3 LLM-Juge local avec auto-calibration.	5
1.4 Plan du mémoire	5
1.4.0.0.1 Note sur l'état de rédaction.	6
2 État de l'art	7
2.1 Intent-Based Networking : fondations	7
2.1.1 Définition : intents, IBN et héritage autonome	7
2.1.1.0.1 Intent réseau.	8
2.1.1.0.2 Intent-Based Networking (IBN).	8
2.1.1.0.3 Héritage autonome et normalisation.	9

2.1.2	Domaines d'application : RAN, slices 5G, sécurité, cœur réseau	9
2.1.2.0.1	Réseau d'accès radio (RAN).	10
2.1.2.0.2	Tranches de réseau (slices) 5G.	10
2.1.2.0.3	Sécurité réseau.	10
2.1.2.0.4	Cœur de réseau (core).	11
2.1.3	Limites des approches IBN classiques	11
2.1.3.0.1	Couverture lexicale limitée.	11
2.1.3.0.2	Fragilité face aux intents nouveaux (<i>brittleness</i>).	12
2.1.3.0.3	Absence d'évaluation automatique de la qualité du traitement.	12
2.2	LLM appliqués aux réseaux (panorama)	13
2.2.1	Génération de configurations	13
2.2.1.0.1	NetConfEval : un banc d'essai de référence.	13
2.2.1.0.2	Adaptation par fine-tuning spécialisé : NetLLM.	14
2.2.1.0.3	Synthèse guidée de configurations de routeurs.	14
2.2.1.0.4	Approches multimodales et augmentées par récupération.	14
2.2.2	Diagnostic et analyse de logs	15
2.2.2.0.1	Hermes : chaînes d'agents pour les réseaux autonomes.	15
2.2.2.0.2	ChatNet : repère académique pour le diagnostic multi-agents.	15
2.2.2.0.3	Cartographie consolidée.	15
2.2.3	Traitement d'intents en langage naturel	16
2.2.3.0.1	Extraction d'intents structurés pour le cœur 5G.	16
2.2.3.0.2	Routage sémantique pour la gestion intent-driven 5G.	16
2.2.3.0.3	Configuration de services intent-driven (Eurecom) et traduction inter-vendeur (INTA).	16
2.3	Routage et sélection dynamique de LLM	17
2.3.1	Approches centrées modèle (model-centric)	18
2.3.1.0.1	RouteLLM : routage à partir de préférences humaines.	18
2.3.1.0.2	Zooter : distillation de reward model en routeur léger.	19
2.3.1.0.3	RouterDC : apprentissage contrastif dual.	19
2.3.1.0.4	Tryage : routeur "perceptif" inspiré du thalamus.	19

2.3.2	Approches centrées requête (query-centric)	20
2.3.2.0.1	FrugalGPT : référence sur le compromis coût-qualité.	20
2.3.2.0.2	Hybrid LLM : routage paramétré par un seuil de qualité.	21
2.3.2.0.3	AutoMix : auto-vérification et escalade dynamique.	21
2.3.2.0.4	LLM-Blender : ensembling, contrepoint au routage.	21
2.3.2.0.5	Eagle : routeur training-free à base d'Elo.	22
2.3.2.0.6	Routage à partir de benchmarks publics.	22
2.3.3	Taxonomie révisée	23
2.3.3.0.1	Axe 1 : Granularité de décision.	23
2.3.3.0.2	Axe 2 : Source d'information.	23
2.3.3.0.3	Axe 3 : Adaptation dans le temps.	24
2.4	Évaluation automatique de qualité (LLM-as-a-Judge)	25
2.4.1	Méthodes classiques (BLEU, ROUGE, BERTScore)	26
2.4.1.0.1	BLEU : métrique à base de n-grammes.	26
2.4.1.0.2	ROUGE : métrique <i>recall-oriented</i> .	26
2.4.1.0.3	BERTScore : métrique sémantique contextuelle.	26
2.4.1.0.4	Limites partagées pour le cas réseau.	27
2.4.2	LLM-as-a-Judge (G-Eval, RocketEval ICLR 2025)	27
2.4.2.0.1	G-Eval : premier framework LLM-as-a-Judge à large diffusion.	28
2.4.2.0.2	Zheng <i>et al.</i> : évaluer le juge lui-même.	28
2.4.2.0.3	JudgeLM : juge spécialisé par fine-tuning.	29
2.4.2.0.4	PandaLM : juge open-source reproductible.	29
2.4.2.0.5	RocketEval : évaluation par checklist instanciée.	30
2.5	Synthèse et positionnement	31
2.5.1	Comparatif des routers existants	31
2.5.1.0.1	Lecture par type de routage.	32
2.5.1.0.2	Lecture par domaine.	32
2.5.1.0.3	Lecture par mécanisme d'évaluation automatique.	32
2.5.1.0.4	Lecture par contraintes coût-latence.	33
2.5.2	Gap identifié et justification d'InferRouter-LLM	33

2.5.2.0.1	Le triple verrou non résolu.	33
2.5.2.0.2	Pertinence opérationnelle pour les opérateurs réseau.	34
2.5.2.0.3	Les trois contributions d’InferRouter-LLM.	34
2.5.2.0.4	Contribution C1 : Estimateur de complexité sémantique d’intents réseau.	35
2.5.2.0.5	Contribution C2 : Routage tri-critère sous contraintes dures.	35
2.5.2.0.6	Contribution C3 : Évaluation automatique par LLM-Juge local.	35
3	Définition du problème	36
3.1	Modélisation (architecture) système	37
3.1.1	Le routeur comme composant central	38
3.1.2	Pool de LLM génériques et spécialisés	38
3.1.3	LLM Juge local	38
3.1.4	Déploiement edge et co-location	40
3.1.5	Comportement attendu par slice réseau	41
3.2	Challenges du système	42
3.2.1	Nombre d’entités impliquées	42
3.2.2	Profondeur d’inférence requise	43
3.2.3	Domaines croisés	43
3.2.4	Opérationnalisation	44
3.3	Formulation du problème	44
3.3.1	Modélisation d’un intent réseau	44
3.3.2	Pool de modèles cibles	45
3.3.3	Critères d’admissibilité	46
3.3.4	Objectif du routeur	46
3.3.4.0.1	Cas dégénéré.	47
3.3.4.0.2	Résolution des égalités.	47
3.3.5	Hypothèses opérationnelles	47
3.3.5.0.1	H1 – Spécialisation par domaine.	47
3.3.5.0.2	H2 – Absence d’oracle humain au runtime.	47
3.3.5.0.3	H3 – Charge système non observable en cloud.	48

3.3.5.0.4	H4 – Indépendance des intents.	48
3.3.6	Qualité agrégée (AIQ)	48
3.3.7	Latence bout en bout (P50, P99)	49
3.3.8	Coût agrégé proxy	49
3.3.9	Distribution de routage	49
3.3.10	Critère principal de comparaison	50
4	InferRouter-LLM	51
4.1	Vue d'ensemble de l'architecture	51
4.2	Contribution 1 : estimateur de complexité sémantique	53
4.3	Contribution 2 : LLM cibles spécialisés par domaine réseau	53
4.4	Contribution 3 : LLM Juge (RocketEval)	54
4.5	Auto-calibration des seuils	54
5	Évaluation	56
5.1	Protocole expérimental	57
5.1.1	Jeu de données d'intents	57
5.1.2	Modèles, juge et métriques	59
5.2	Estimateur de complexité sémantique	60
5.2.1	Un confond de longueur trompeur	60
5.2.2	Mesure honnête à longueur contrôlée	60
5.2.3	Embeddings de phrase : un signal réévalué	61
5.3	Fiabilité du LLM Juge	61
5.3.1	Checklist fixe contre RocketEval complet	61
5.3.2	Discrimination grossière contre discrimination fine	62
5.4	Prémisse du routage et calibration	63
5.4.1	Matrice de qualité par tier et complexité	63
5.4.2	Un léger plus compétitif	64
5.5	Synthèse et limites	64
6	Conclusion	66
6.1	Synthèse	66

CONTENTS

6.2	Apports scientifiques	66
6.3	Limites et perspectives	66
	Bibliography	67
	Appendices	
A	Annexes techniques	74
A.1	Configuration du système	74
A.2	Exemples d'intents et réponses	74
A.2.1	Un intent annoté du jeu de données	74
A.2.2	Checklist RocketEval générée pour cet intent	75

List of Tables

2.1	Positionnement des principaux travaux de routage de LLM selon la taxonomie à trois axes.	25
2.2	Comparaison des routers LLM existants face a InferRouter-LLM	31
3.1	Notations du cadre formel d’InferRouter-LLM.	37
3.2	Répercussion du type de slice sur la criticité $\kappa(e)$ et les bornes de l’intent (régimes qualitatifs dérivés des exigences de service de la spécification 3GPP TS 22.261 [46]). .	42
5.1	Attributs d’un intent du jeu de données : valeurs possibles et méthode d’attribution. .	57
5.2	Grille d’annotation de la complexité, exprimée sur les trois critères du chapitre 3. Au niveau complexe, le croisement de domaines est majoritaire mais non systématique. . .	58
5.3	Modèles de la campagne : rôle, taille et tarif unitaire c_i en USD pour 1000 jetons (grille OpenRouter en vigueur au moment des expériences). n.c. : taille non communiquée par l’éditeur ; n.d. : tarif non consigné. Les juges locaux s’exécutent via Ollama, sans coût d’appel.	59
5.4	Exactitude de l’estimateur de complexité par variante du jeu de données et par signal (validation croisée $k = 5$ sur 252 intents, régression logistique sauf mention RF, baseline majoritaire 33,3%).	60
5.5	Accord du LLM Juge selon le régime de discrimination et la configuration. Grossier : bon candidat contre mauvais. Fin : réponse correcte contre variante à une erreur injectée. .	63
5.6	Qualité moyenne notée par le juge, par tier de modèle et niveau de complexité (échantillon de calibration, 12 intents équilibrés).	63

Chapter 1

Introduction

Contents

1.1	Contexte et motivation	2
1.2	Problématique	3
1.3	Contributions	4
1.4	Plan du mémoire	5

1.1 Contexte et motivation

L'adoption des grands modèles de langage (Large Language Models (LLMs)) en milieu industriel a franchi un seuil majeur sur la période 2023-2024. Selon le rapport AI Index 2025 de l'Institute for Human-Centered AI de l'Université de Stanford, 78 % des organisations déclarent recourir à l'intelligence artificielle dans au moins une fonction métier en 2024, contre 55 % un an plus tôt [1]. Dans le même intervalle, le coût d'inférence des LLM a chuté d'un facteur d'environ 280, grâce à la conjonction de modèles plus compacts, de techniques de quantification et d'optimisations d'exécution [1]. Cette double dynamique d'adoption massive et de baisse drastique des coûts unitaires déplace la frontière des usages : les LLM sortent du périmètre des applications conversationnelles généralistes pour s'établir comme briques opérationnelles de domaines verticaux exigeants, dont les télécommunications [2].

Le secteur réseau aborde simultanément une transition opérationnelle inédite. La densification des accès radio, la virtualisation des fonctions, le network slicing 5G et la multiplication des indicateurs de qualité à superviser confrontent les opérateurs à une complexité de gestion qui dépasse les capacités

1.2. PROBLÉMATIQUE

d’une administration purement humaine [3]. Le passage à la 6G accentue ce phénomène : la diversité attendue des services et la concurrence entre politiques de qualité de service rendent les conflits d’objectifs inévitables et imposent un raisonnement automatique au-delà des seuils classiques [4]. Ces constats motivent une nouvelle génération de systèmes d’automatisation, capables de raisonner sur l’intention de l’opérateur plutôt que sur des recettes de configuration manuelles.

L’Intent-Based Networking (Intent-Based Networking (IBN)) constitue la réponse paradigmatique à ce besoin. La RFC 9315 de l’Internet Research Task Force (IRTF) formalise un intent comme un ensemble d’objectifs et de résultats déclaratifs que le réseau doit satisfaire, sans spécifier la manière de les atteindre [5]. Le paradigme est également documenté en milieu industriel : Falkner et Apostolopoulos décrivent l’IBN comme l’architecture de référence pour les réseaux d’entreprise modernes et identifient la phase de traduction (“translation”) de l’intent en configuration exécutable comme l’étape opérationnellement la plus fragile [6]. C’est précisément à cette phase que les LLM, dotés de capacités de compréhension du langage naturel, apparaissent comme des candidats techniques [7, 8].

L’usage des LLM pour la translation d’intents réseau ouvre néanmoins une série de questions opérationnelles non triviales. Un modèle unique ne couvre que médiocrement la diversité des domaines techniques sous-jacents (accès radio, cœur, sécurité, slicing), ce qui motive le recours à un ensemble de modèles spécialisés [2, 9]. L’inférence à l’échelle d’un opérateur engage des coûts significatifs qui justifient un arbitrage économique systématique [10]. La latence ajoutée par un appel LLM doit rester compatible avec les boucles d’assurance des réseaux. Enfin, l’évaluation de la qualité d’une réponse à un intent réseau ne peut reposer durablement sur une supervision humaine à grande échelle. Ces quatre tensions (spécialisation, coût, latence et qualité vérifiable) constituent le point de départ du présent travail.

1.2 Problématique

L’état de l’art du routage de LLM et de l’application des LLM aux réseaux montre que les axes de spécialisation, de routage multi-critère et d’évaluation automatique sont aujourd’hui traités de manière séparée. Les approches de routage économique [10] concentrent l’arbitrage sur le couple coût-qualité sans contrainte de latence par intent — c’est-à-dire évaluée intent par intent. Les travaux sur les LLM appliqués aux réseaux [9, 2] démontrent la valeur d’une spécialisation par domaine sans formaliser le

1.3. CONTRIBUTIONS

mécanisme de sélection. Hong identifie explicitement le routage et la spécialisation comme verrous ouverts pour le passage à l'échelle opérateur [11]. À notre connaissance, aucun travail ne combine simultanément (a) une spécialisation par domaine réseau, (b) un routage tri-critère sous contraintes dures de coût et de latence évaluées par intent et (c) une évaluation automatique de qualité à la fois fiable et économique (cf. §2.5.2).

Ce manque est problématique pour les opérateurs réseau. Les volumes d'intents traités quotidiennement, les engagements de service contractuels et les budgets d'inférence alloués par client interdisent une approche fondée sur un modèle unique ou sur une supervision humaine systématique. L'absence d'un cadre conjoint coût-latence-qualité produit en pratique des systèmes qui optimisent un critère au détriment des autres et qui restent dépendants d'une boucle d'évaluation humaine non passable à l'échelle.

Le présent mémoire s'articule autour de la question de recherche suivante : “comment router dynamiquement des intents réseau hétérogènes vers des LLM spécialisés en respectant des contraintes dures de coût et de latence par intent, sans humain dans la boucle pour évaluer la qualité ?” Cette question fixe simultanément la portée du travail (intents réseau, pas requêtes génériques), son cadre formel (contraintes dures évaluées par intent) et son exigence opérationnelle (autonomie de l'évaluation).

1.3 Contributions

Pour répondre à la question posée, le présent travail introduit InferRouter-LLM, un système de routage d'intents réseau articulé autour de trois contributions complémentaires.

1.3.0.0.1 Estimateur sémantique de complexité d'intents réseau. La première contribution est un estimateur de la complexité d'un intent fondé sur des représentations vectorielles produites par un modèle `sentence-transformers`, suivies d'un classifieur supervisé léger réalisé sous scikit-learn. L'estimateur est conçu pour une exécution en moins de 15 ms sur l'environnement cible et est calibré sur un corpus d'intents réseau généré par LLM, couvrant les quatre domaines fonctionnels et trois niveaux de complexité (chapitre 5). Les critères de complexité formels sont définis au chapitre 3 (§3.2) : nombre d'entités réseau mobilisées, profondeur des contraintes, diversité des domaines techniques sollicités. Le détail d'implémentation est exposé au chapitre 4 (4).

1.3.0.0.2 Routage tri-critère sous contraintes dures par intent. La deuxième contribution est la formalisation explicite du problème de routage comme la sélection d’un modèle dans un ensemble admissible défini par intent. Pour un intent e , l’ensemble des modèles admissibles s’écrit

$$M(e) = \{ m_i \in M : \ell_i(e) \leq L_{\max}(e) \wedge c_i \leq C_{\max}(e) \}$$

où $\ell_i(e)$ désigne la latence prédite du modèle m_i sur l’intent e , c_i son coût unitaire, et $L_{\max}(e)$ et $C_{\max}(e)$ les bornes dures admises pour cet intent (cf. (3.5)). Le routeur choisit alors le modèle qui maximise la qualité attendue $\hat{q}_i(e)$, estimation a priori de la qualité calibrée par un LLM-Juge (chapitre 3), c’est-à-dire $R^*(e) = \arg \max_{m_i \in M(e)} \hat{q}_i(e)$ (cf. (3.6)). À notre connaissance, cette formalisation conjointe coût-latence-qualité avec bornes évaluées par intent constitue une formulation inédite dans le domaine du routage de LLM appliqué aux réseaux. Hong, dans son survey de 2025, identifie explicitement le routage comme verrou ouvert pour le passage à l’échelle opérateur [11].

1.3.0.0.3 LLM-Juge local avec auto-calibration. La troisième contribution est l’intégration d’un LLM-Juge exécuté localement (gemma2 via Ollama) pour évaluer automatiquement la qualité des réponses produites par les modèles cibles. Le juge applique la méthode RocketEval [12], fondée sur des listes de vérification (checklists)instanciées par intent, qui rapporte selon ses auteurs une corrélation de Spearman de l’ordre de 0,965 avec les préférences humaines sur le benchmark MT-Bench, pour un coût marginal quasiment nul à l’inférence [12]. L’exécution locale du juge élimine la dépendance à une Application Programming Interface (API) cloud propriétaire et alimente la calibration des seuils du routeur sans intervention humaine. Notre évaluation (chapitre 5) circonscrit l’usage de ce juge : fiable pour départager des candidats nettement différents, donc pour le routage, mais pas comme oracle d’une qualité absolue fine. Les fondations méthodologiques du paradigme LLM-as-a-Judge sont détaillées en §2.4.2.

Ces trois contributions sont décrites en détail au chapitre 4 (4) et évaluées expérimentalement au chapitre 5 (5).

1.4 Plan du mémoire

Le mémoire est organisé en six chapitres dont le présent constitue l’introduction. Le chapitre 2 (2) développe l’état de l’art en cinq sections : fondations de l’Intent-Based Networking, applications des

1.4. PLAN DU MÉMOIRE

LLM aux réseaux, routage et sélection dynamique de LLM, évaluation automatique selon le paradigme LLM-as-a-Judge, puis synthèse et positionnement par rapport à la littérature. Le chapitre 3 (3) formalise le problème traité : notation des intents et du pool de modèles, hypothèses de travail, critères de complexité d'un intent réseau, métriques de succès. Le chapitre 4 (4) présente l'architecture d'InferRouter-LLM et détaille chacune des trois contributions ainsi que leur articulation. Le chapitre 5 (5) rapporte l'évaluation expérimentale menée sur le jeu de données d'intents généré par LLM, couvre les métriques de qualité, de coût et de latence, et inclut les études d'ablation. Le chapitre 6 (6) conclut en résumant les apports scientifiques, en discutant les limites de l'approche et en ouvrant des perspectives.

1.4.0.0.1 Note sur l'état de rédaction. Les chapitres 4, 5 et 6 sont actuellement en cours de rédaction. La présente version partielle du mémoire est soumise pour validation de la nouvelle orientation avant la phase expérimentale.

Chapter 2

État de l’art

Contents

2.1	Intent-Based Networking : fondations	7
2.1.1	Définition : intents, IBN et héritage autonome	7
2.1.2	Domaines d’application : RAN, slices 5G, sécurité, cœur réseau	9
2.1.3	Limites des approches IBN classiques	11
2.2	LLM appliqués aux réseaux (panorama)	13
2.2.1	Génération de configurations	13
2.2.2	Diagnostic et analyse de logs	15
2.2.3	Traitement d’intents en langage naturel	16
2.3	Routage et sélection dynamique de LLM	17
2.3.1	Approches centrées modèle (model-centric)	18
2.3.2	Approches centrées requête (query-centric)	20
2.3.3	Taxonomie révisée	23
2.4	Évaluation automatique de qualité (LLM-as-a-Judge)	25
2.4.1	Méthodes classiques (BLEU, ROUGE, BERTScore)	26
2.4.2	LLM-as-a-Judge (G-Eval, RocketEval ICLR 2025)	27
2.5	Synthèse et positionnement	31
2.5.1	Comparatif des routers existants	31
2.5.2	Gap identifié et justification d’InferRouter-LLM	33

2.1 Intent-Based Networking : fondations

2.1.1 Définition : intents, IBN et héritage autonome

La présente section pose les définitions canoniques sur lesquelles repose l’ensemble du mémoire. L’objet manipulé par InferRouter-LLM est l’*intent réseau* ; il convient donc d’en circonscrire précisément

le sens, par opposition aux notions voisines de politique (“policy”) et de modèle de service (“service model”).

2.1.1.0.1 Intent réseau. La RFC 9315 de l’IRTF définit un intent comme “un ensemble d’objectifs opérationnels qu’un réseau doit satisfaire et de résultats qu’il doit produire, exprimés de manière déclarative sans spécifier comment les atteindre” [5]. Cette définition implique deux propriétés distinctives. D’une part, l’intent est déclaratif : il énonce un résultat attendu (ce que le réseau doit accomplir) et non une procédure de configuration (comment y parvenir). D’autre part, il est formulé à un haut niveau d’abstraction, typiquement compréhensible par un opérateur humain sans connaissance des artefacts de configuration sous-jacents (commandes CLI, primitives YANG, manifestes NETCONF). À ce double titre, l’intent se distingue d’une politique classique, qui prescrit généralement un comportement *event-condition-action* explicite [5], ainsi que d’un modèle de service, qui décrit la structure d’un service offert mais ne spécifie pas nécessairement les objectifs opérationnels associés.

Dans la suite, nous adoptons la notation $e \in E$ pour désigner un intent appartenant à l’ensemble fini d’intents traités par le système, conformément à la formalisation du chapitre 3 (cf. §3.2).

2.1.1.0.2 Intent-Based Networking (IBN). L’IBN désigne le paradigme opérationnel qui prend l’intent comme primitive d’interaction entre l’opérateur et le réseau. Un Intent-Based Networking System (IBNS), selon la taxonomie de Leivadeas et Falkner [7], articule cinq composantes formant une boucle fermée d’automatisation :

1. *intent expression* : saisie de l’intent par l’opérateur, en langage naturel ou via une interface guidée ;
2. *intent translation* : transformation de l’intent déclaratif en une représentation intermédiaire formelle, exploitable par les couches de configuration ;
3. *intent resolution* : détection et résolution des conflits entre intents concurrents ou avec l’état courant du réseau ;
4. *intent activation* : déploiement effectif de la configuration résultante sur les équipements (physiques ou virtualisés) ;

2.1. INTENT-BASED NETWORKING : FONDATIONS

5. *intent assurance* : vérification continue que les résultats observés respectent l'intent initial, avec rétroaction corrective.

La RFC 9315 énonce par ailleurs six principes opératoires structurant le fonctionnement d'un IBNS : source unique de vérité, raffinement itératif, autonomie supervisée, apprentissage continu, exposition de capacités et abstraction [5]. Le présent travail se concentre sur la phase de translation, identifiée comme étape critique par Pang *et al.* [13] et Mehmood *et al.* [8] : c'est en effet à ce stade que la diversité lexicale des intents formulés par les opérateurs entre en collision avec la rigidité des langages de configuration cibles.

2.1.1.0.3 Héritage autonome et normalisation. L'IBN s'inscrit dans la filiation de l'*autonomic networking*, paradigme initié par IBM dans les années 2000 et formalisé ultérieurement au sein du groupe Autonomic Networking Integrated Model and Approach (ANIMA) de l'Internet Engineering Task Force (IETF). La continuité conceptuelle est explicite : "l'IBN constitue l'évolution moderne des réseaux autonomes, tirant parti des avancées en apprentissage automatique pour rendre opérationnelle l'auto-gestion théorisée par l'autonomic computing" [7]. Sur le plan normatif, plusieurs organismes contribuent à la maturation industrielle du paradigme : l'IRTF via la RFC 9315 [5], le Third Generation Partnership Project (3GPP) via les spécifications de gestion 5G/6G, et l'European Telecommunications Standards Institute (ETSI) via le groupe Zero-touch network and Service Management (ZSM) dont le rapport "GR ZSM 011" définit les "intent-driven autonomous networks" comme axe structurant des architectures zero-touch [14]. Mehmood *et al.* [8] dressent une synthèse exhaustive de ces standards en y associant les contributions du TM Forum (TMF), confirmant la convergence des feuilles de route industrielles vers le paradigme IBN. Les quatre surveys de référence retenus pour ce mémoire [13, 15, 7, 8] fournissent un panorama cohérent : ils s'accordent sur la maturité conceptuelle de l'IBN tout en pointant le même verrou pratique, à savoir la traduction robuste d'intents en langage naturel vers des configurations exécutables. C'est ce verrou qui motive directement le présent travail.

2.1.2 Domaines d'application : RAN, slices 5G, sécurité, cœur réseau

L'IBN ne constitue pas un paradigme monolithique : ses concrétisations varient sensiblement selon le sous-système réseau ciblé. Suivant la cartographie proposée par Mehmood *et al.* [8] et complétée par Leivadeas et Falkner [7], nous distinguons quatre domaines d'application principaux qui structureront,

dans la suite du mémoire, l'ensemble $\mathcal{D}$ des spécialisations considérées par le routeur (cf. chapitre 3).

2.1.2.0.1 Réseau d'accès radio (Radio Access Network (RAN)). Les intents appliqués au RAN portent typiquement sur l'optimisation des paramètres radio (puissance d'émission, configuration de beamforming, choix de Modulation and Coding Scheme (MCS)) ainsi que sur le placement de cellules et l'équilibrage de charge entre porteurs. Pang *et al.* [13] identifient le RAN comme l'un des domaines pionniers de l'IBN, en raison du caractère quantifiable des objectifs (couverture, débit, interférence) qui se prêtent naturellement à une formulation déclarative. Mehmood *et al.* [8] mettent en évidence que l'introduction du RAN Intelligent Controller (RIC) dans les architectures O-RAN offre un point d'ancrage concret pour l'activation d'intents radio, via les applications *xApps* et *rApps*.

2.1.2.0.2 Tranches de réseau (slices) 5G. Le *network slicing* constitue un terrain d'application particulièrement actif pour l'IBN [15]. Un intent typique spécifie une classe de service (Ultra-Reliable Low-Latency Communications (uRLLC), enhanced Mobile BroadBand (eMBB) ou massive Machine Type Communications (mMTC)) assortie de garanties Service Level Agreement (SLA) (latence, fiabilité, débit) et doit être traduit en provisionnement cohérent d'une Network Slice Instance (NSI) traversant les segments d'accès, de transport et de cœur. Mehmood *et al.* [8] soulignent que l'isolation entre tranches et la gestion des conflits inter-slices représentent deux défis ouverts particulièrement bien adressés par l'approche intent-driven, parce que la déclarativité facilite le raisonnement multi-domaine.

2.1.2.0.3 Sécurité réseau. L'application de l'IBN à la sécurité, parfois désignée *Intent-Based Security* ou Intent-Based Security (IBSec), est un domaine émergent. Les intents typiques portent sur l'expression de politiques d'isolation (Access Control List (ACL), segmentation), la configuration de pare-feu distribués et le paramétrage de systèmes de détection d'intrusion (Intrusion Detection System (IDS)). Leivadeas et Falkner [7] notent que la sécurité bénéficie particulièrement de la formulation déclarative, dans la mesure où un même objectif de protection peut s'exprimer indépendamment de la topologie sous-jacente et être re-traduit dynamiquement lorsque celle-ci évolue. Mehmood *et al.* [8] précisent toutefois que la traduction d'intents de sécurité demeure largement manuelle dans les plateformes industrielles actuelles, faute d'outillage de vérification formelle mature.

2.1.2.0.4 Cœur de réseau (core). Les intents appliqués au cœur 5G visent la configuration cohérente des fonctions Access and Mobility Management Function (AMF), Session Management Function (SMF) et User Plane Function (UPF), ainsi que la gestion des règles de Quality of Service (QoS) et des politiques portées par le Policy Control Function (PCF) [8]. La complexité provient ici de l'enchevêtrement des dépendances entre fonctions virtualisées : un intent unique peut requérir la reconfiguration coordonnée de plusieurs Network Function (NF). Zeydan et Turk [15] citent les plateformes industrielles Cisco DNA, Apstra et Juniper Apstra comme illustrations de cette approche, en notant qu'elles s'appuient principalement sur des langages dédiés (NEMO, Network Intent Composition d'OpenDaylight) au détriment d'une interface en langage naturel.

Ces quatre domaines forment la taxonomie sur laquelle s'appuie l'estimateur de complexité formalisé en §3.2 : la difficulté intrinsèque d'un intent dépend non seulement de sa surface lexicale, mais aussi du domaine cible et du nombre de frontières inter-domaines qu'il franchit. La spécialisation des LLM cibles par domaine, retenue comme contribution secondaire du système, prend racine dans cette segmentation.

2.1.3 Limites des approches IBN classiques

Malgré la maturité conceptuelle du paradigme et la convergence des standards, les réalisations opérationnelles d'IBN antérieures aux modèles de langage de grande taille restent confrontées à trois limites récurrentes, identifiées de manière convergente par les surveys [13, 15, 7, 8] et illustrées par les systèmes d'époque.

2.1.3.0.1 Couverture lexicale limitée. Les approches d'"expression d'intent" antérieures aux LLM reposent typiquement sur une combinaison de Natural Language Processing (NLP) à base de règles, d'ontologies (par exemple Web Ontology Language (OWL)) et de grammaires spécialisées. Le système LUMI de Jacobs *et al.* [16] en constitue l'archetype académique : il combine Named Entity Recognition (NER) classique et règles de transformation pour traduire des intents en langage naturel vers le langage formel NILE, et nécessite une boucle de feedback humain pour résoudre les ambiguïtés. Évalué sur des intents extraits de 50 réseaux campus, Lumi atteint des performances satisfaisantes sur des formulations canoniques, mais peine à généraliser aux paraphrases naturelles et aux formulations ambiguës ne figurant pas dans sa base d'apprentissage [16]. Plus généralement, Zeydan et Turk [15] soulignent

qu’“avant l’arrivée des progrès récents en NLU, l’écosystème IBN était mûr conceptuellement mais limité par les techniques NLP classiques”, faisant des avancées en traitement du langage la “brique manquante” pour passer de l’IBN-théorique à l’IBN-opérationnel.

2.1.3.0.2 Fragilité face aux intents nouveaux (*brittleness*). Une seconde limite, plus structurelle, tient à la rigidité des artefacts spécialisés. Toute introduction d’un nouveau type d’intent (nouvelle classe de service, nouvelle politique de sécurité, nouveau Key Performance Indicator (KPI) à optimiser) requiert l’ajout manuel d’un patron syntaxique, d’une entrée ontologique ou d’une règle de transformation supplémentaire dans le Domain-Specific Language (DSL) cible. Cette charge de maintenance croît linéairement avec la couverture fonctionnelle visée, ce que Leivadeas et Falkner [7] identifient comme l’un des principaux freins industriels à l’adoption massive de l’IBN. Mehmood *et al.* [8] formalisent ce phénomène comme un défi de “scalabilité sémantique” : la généralisation à une longue queue d’intents hétérogènes constitue un verrou explicite des architectures pré-LLM.

2.1.3.0.3 Absence d’évaluation automatique de la qualité du traitement. La troisième limite concerne la phase d’*intent assurance*. Dans les systèmes classiques, la vérification que la traduction $\text{intent} \rightarrow \text{configuration}$ est sémantiquement correcte repose soit sur une validation humaine experte, soit sur l’exécution de tests fonctionnels coûteux en environnement de préproduction [13]. Aucun mécanisme générique ne permet, à partir du seul couple (intent, configuration générée), d’émettre un jugement de qualité automatique ; les métriques classiques de génération (BLEU, ROUGE) sont par ailleurs inadaptées au cas des configurations réseau, qui exigent une correction fonctionnelle plutôt qu’une proximité textuelle. Cette absence d’évaluation systématique constitue “l’un des freins majeurs au déploiement closed-loop” selon Pang *et al.* [13], et fait écho aux préoccupations exprimées dans le cadre des architectures zero-touch ZSM [14].

Ces trois limites (couverture lexicale, fragilité face à la nouveauté et carence d’évaluation automatique) définissent le cahier des charges implicite que les approches récentes fondées sur les LLM ambitionnent de satisfaire. La section suivante (§2.2) examine précisément comment les LLM ont été mobilisés pour lever ces verrous, ouvrant la voie à la problématique du routage entre modèles traitée en §2.3.

2.2 LLM appliqués aux réseaux (panorama)

L'arrivée des LLM généralistes à partir de 2023 a ouvert une vague de travaux dans laquelle la communauté réseau a cherché à transposer les capacités de compréhension et de génération de texte de ces modèles aux tâches opérationnelles laissées ouvertes par l'IBN classique (cf. §2.1.3). Deux surveys de référence, Liu *et al.* [17] et Zhou *et al.* [18], cartographient ce champ émergent et convergent sur une partition en trois grandes familles applicatives : (i) la génération de configurations réseau à partir d'énoncés de haut niveau ; (ii) le diagnostic et l'analyse de journaux opérationnels ; et (iii) le traitement d'intents formulés en langage naturel. La présente section parcourt ces trois familles en s'appuyant sur les travaux les plus structurants de la période 2023–2025. L'objectif n'est pas de dresser un inventaire exhaustif (les surveys cités ci-dessus s'en chargent), mais d'extraire trois constats convergents qui motivent directement la problématique du routage traitée en §2.3 : l'hétérogénéité des tâches réseau [17], la sensibilité des LLM généralistes aux spécificités du domaine [18, 19], et l'usage quasi-systématique d'un modèle cloud propriétaire unique sans considération explicite du compromis coût-qualité-latence [9, 20].

2.2.1 Génération de configurations

La première famille de travaux explore la capacité des LLM à produire des configurations d'équipements réseau (typiquement des fichiers Cisco IOS, Juniper Junos ou des manifestes NETCONF) à partir d'énoncés de haut niveau, qu'il s'agisse d'exigences fonctionnelles, de patrons de politique ou d'extraits documentaires. Cette famille constitue le banc d'essai le plus ancien et le plus structuré du panorama, car la tâche se prête à une évaluation automatisée par vérificateurs symboliques.

2.2.1.0.1 NetConfEval : un banc d'essai de référence. L'étude conduite par Wang *et al.* [21], publiée aux *Proceedings of ACM on Networking* (CoNEXT 2024) et récompensée par le *Applied Networking Research Prize* de l'IRTF, propose le benchmark NETCONFVAL, qui évalue plusieurs LLM (GPT-3.5, GPT-4, Code Llama) sur quatre tâches graduées de difficulté croissante : (i) traduction d'exigences exprimées en langage naturel vers une spécification formelle, (ii) génération d'appels API et de fonctions, (iii) implémentation d'algorithmes de routage, et (iv) génération de configurations bas-niveau pour des protocoles tels que BGP ou OSPF [21]. L'apport méthodologique majeur de NetConfEval est de montrer que la qualité des sorties LLM dépend non linéairement de la complexité

2.2. LLM APPLIQUÉS AUX RÉSEAUX (PANORAMA)

syntactique des artefacts attendus : les modèles réussissent mieux la traduction d'exigences vers une représentation intermédiaire formelle que la génération directe de configurations bas-niveau, dont la fragilité augmente avec le nombre de fonctionnalités couplées. Ce constat fonde directement la justification d'un estimateur de complexité, brique centrale du présent travail. Il est notable que ce papier ait été explicitement transmis par le tuteur du mémoire comme référence canonique : il fixe à la fois le cadre méthodologique et le vocabulaire d'évaluation que nous reprendrons au chapitre 5.

2.2.1.0.2 Adaptation par fine-tuning spécialisé : NetLLM. Wu *et al.* [19] proposent à SIGCOMM 2024 le framework NETLLM, première tentative systématique d'adapter un LLM généraliste aux données réseau hétérogènes via une procédure d'adaptation à faible coût transposant les principes de Low-Rank Adaptation (LoRA) [19], évaluée sur trois tâches (prédiction de *viewport* 360°, contrôle adaptatif du débit Adaptive Bitrate (ABR), ordonnancement de tâches). NetLLM incarne la position opposée à celle d'InferRouter-LLM : au lieu de router une requête vers un LLM spécialisé, les auteurs adaptent un même LLM à toutes les tâches ; ce contraste sera exploité en §2.5 pour positionner notre contribution.

2.2.1.0.3 Synthèse guidée de configurations de routeurs. À HotNets 2023, Mondal *et al.* [22] montrent qu'invoqué seul, GPT-4 produit des configurations affectées d'erreurs structurelles (incompréhension de topologie, erreurs syntaxiques Cisco IOS ou Junos, violations sémantiques de politiques) et proposent en réponse le paradigme *Verified Prompt Programming* couplant le LLM à un vérificateur symbolique externe avec boucle de feedback localisée, illustré par la traduction Cisco → Juniper et la mise en œuvre d'une politique *no-transit* multi-routeurs [22]. Ce travail établit deux résultats que nous reprenons : la fragilité intrinsèque des LLM sur la génération de configuration, qui motive le besoin d'évaluation automatique abordé en §2.4 ; et l'efficacité d'une architecture hybride associant un LLM à un composant externe de validation, principe sur lequel s'appuie notre LLM Juge.

2.2.1.0.4 Approches multimodales et augmentées par récupération. Enfin, Ifland *et al.* [23] présentent GENET, co-pilote multimodal combinant la lecture d'une topologie sous forme de diagramme et un énoncé textuel d'intent pour générer les configurations device-level. Évalué sur des exercices de certification Cisco CCNA, GeNet illustre l'apport potentiel de la récupération documentaire (Retrieval-Augmented Generation (RAG)) ; cette piste reste hors périmètre direct d'InferRouter-LLM, qui se limite à l'entrée textuelle, et sera évoquée en perspectives (chapitre 6).

2.2. LLM APPLIQUÉS AUX RÉSEAUX (PANORAMA)

Ces travaux convergent vers un constat partagé : la génération de configurations par LLM atteint une qualité acceptable sur les tâches bien spécifiées et de complexité modérée, mais reste sensible aux ambiguïtés lexicales et aux erreurs sémantiques dès que l’on franchit le seuil de fonctionnalités couplées identifié par NetConfEval [21], justifiant une étape diagnostique préliminaire.

2.2.2 Diagnostic et analyse de logs

La deuxième famille exploite la capacité des LLM à interpréter de grands volumes de texte semi-structuré pour assister l’opérateur dans le diagnostic d’incidents et la corrélation d’événements.

2.2.2.0.1 Hermes : chaînes d’agents pour les réseaux autonomes. Ayed *et al.* [24] proposent HERMES, framework développé par Huawei et Yale orchestrant plusieurs agents LLM via des *blueprints* pour construire des instances de Network Digital Twin (NDT), avec deux rôles séparés (*Designer* et *Coder*) validés sur trois cas d’usage (*slice provisioning*, positionnement et déploiement de services) [24]. Hermes illustre une tendance lourde : passer du LLM unique à une chaîne d’agents spécialisés par rôle ; cette structuration est voisine du routage vers experts défendu par InferRouter-LLM, mais reste orientée rôles fonctionnels plutôt que domaines réseau.

2.2.2.0.2 ChatNet : repère académique pour le diagnostic multi-agents. Huang *et al.* [9] décrivent dans *IEEE Network* le système CHATNET, framework de diagnostic conversationnel composé de quatre rôles dédiés (*analyzer*, *planner*, *calculator*, *executor*), chacun instancié par un appel à un LLM généraliste (GPT-4) augmenté d’outils externes [9]. ChatNet partage avec InferRouter-LLM l’intuition d’un découpage par rôles mais s’en distingue par l’absence de mécanisme explicite de sélection du modèle cible : le même LLM généraliste est invoqué quel que soit le rôle, sans arbitrage coût/latence/qualité.

2.2.2.0.3 Cartographie consolidée. Le survey de Zhou *et al.* [18] cartographie les cas d’usage LLM-pour-télécoms 2023–2024 et identifie trois enjeux ouverts pour le diagnostic : confidentialité des données opérationnelles, latence d’inférence sur de longs contextes, et coût économique de l’invocation répétée d’un modèle cloud propriétaire. Ce dernier enjeu, absent de l’argumentaire de Hermes ou de ChatNet, motive la problématique du routage traitée en §2.3 : les trois systèmes ci-dessus mobilisent

2.2. LLM APPLIQUÉS AUX RÉSEAUX (PANORAMA)

un LLM unique (GPT-3.5 ou GPT-4) sans questionner la pertinence d'un modèle moins coûteux pour les requêtes simples [9, 24, 18].

2.2.3 Traitement d'intents en langage naturel

La troisième famille cible explicitement la phase de translation d'un intent en langage naturel vers une représentation exploitable par les couches de configuration ; c'est celle dans laquelle InferRouter-LLM s'inscrit le plus directement.

2.2.3.0.1 Extraction d'intents structurés pour le cœur 5G. Manias *et al.* [25] traitent à DRCN 2024 l'extraction d'intents 5G structurés à partir d'énoncés libres, en produisant une représentation JavaScript Object Notation (JSON) explicitant les entités (fonction réseau cible, type de slice, KPI attendu) et comparent plusieurs LLM open-source et propriétaires [25]. Cette étude valide empiriquement la pertinence d'un découpage par domaine réseau : les performances d'extraction sont sensiblement plus élevées lorsque le modèle est conditionné par un contexte de domaine explicite (cœur, RAN, slice).

2.2.3.0.2 Routage sémantique pour la gestion intent-driven 5G. La même équipe prolonge ce travail à GLOBECOM 2024 avec une contribution particulièrement proche du présent mémoire [20] : un *semantic router* préfiltre les requêtes opérateur en encodant l'intent via SENTENCE-BERT ou MINILM pour sélectionner le sous-système cible le plus pertinent, améliorant précision globale et coût d'inférence par rapport à une architecture de prompting pure. Une différence structurante apparaît toutefois : chez Manias *et al.*, le routage sémantique désigne le composant réseau (fonction du cœur 5G) auquel adresser une requête ; chez InferRouter-LLM, il désigne le LLM cible le mieux adapté à la complexité estimée de l'intent. Les deux approches sont en réalité complémentaires : l'une oriente vers la cible opérationnelle, l'autre vers l'outil de raisonnement. Cette distinction sera reprise en §2.5 pour caractériser le positionnement de notre contribution.

2.2.3.0.3 Configuration de services intent-driven (Eurecom) et traduction inter-vendeur (INTA).

Mekrache et Ksentini [26] présentent à NetSoft 2024 un système LLM-based traduisant des intents en langage naturel vers des Network Service Descriptor (NSD) exploitables par un orchestrateur de

2.3. ROUTAGE ET SÉLECTION DYNAMIQUE DE LLM

services nouvelle génération, avec boucle *Human Feedback* et démonstration dans le contexte 6G d'Eurecom (OSS-GPT) [26] ; ce système intervient en aval d'un éventuel routeur tel qu'InferRouter-LLM, pour matérialiser l'intent traduit en configuration exécutable. Plus récemment, Wei *et al.* [27] proposent à ICNP 2025 le framework INTA, qui mobilise une chaîne d'agents LLM pour traduire des configurations entre équipements hétérogènes (Cisco → Juniper) en passant par une représentation intermédiaire d'intent, et rapporte une exactitude syntaxique de 98,15 % et un rappel commandes de 84,72 % [27], confirmant la viabilité de l'intent comme représentation intermédiaire commune.

Ces quatre contributions partagent un schéma commun : toutes mobilisent un unique LLM en aval, le plus souvent un modèle cloud propriétaire (GPT-3.5, GPT-4) appelé via API, sans mécanisme explicite d'arbitrage entre plusieurs modèles candidats. Lorsque l'orientation sémantique existe (Manias *et al.* [20]), elle vise un composant opérationnel et non un outil de raisonnement. Cette absence d'un arbitrage tri-critère coût-qualité-latence à la couche "outil de raisonnement" constitue précisément le *gap* adressé par le présent mémoire ; la section suivante en propose une formalisation à partir de la littérature du routage entre LLM.

2.3 Routage et sélection dynamique de LLM

Le panorama de la §2.2 a mis en évidence un verrou méthodologique récurrent : les systèmes existants invoquent quasi-systématiquement un modèle unique, typiquement un LLM cloud propriétaire de la famille GPT, quelle que soit la complexité de la requête traitée [9, 24, 18]. Cette uniformisation ignore un fait empirique pourtant bien documenté dans la littérature plus large des LLM : face à un pool de modèles hétérogènes présentant des compromis distincts entre coût, qualité et latence, il existe pour chaque requête un modèle plus efficient qu'un choix uniforme [10, 28]. La problématique du routage de LLM consiste précisément à apprendre, pour une requête donnée, le modèle cible qui maximise une utilité sous contraintes.

La littérature 2023–2025 sur ce sujet s'organise selon deux familles méthodologiques complémentaires. Les approches centrées modèle (*model-centric*) construisent une représentation des LLM cibles, via leur classement Elo (score de niveau issu de comparaisons par paires, hérité des échecs) sur des arènes humaines, des reward models distillés ou des embeddings appris, puis apprennent un routeur qui projette une requête sur ce référentiel modèle. Les approches centrées requête (*query-centric*) prennent

2.3. ROUTAGE ET SÉLECTION DYNAMIQUE DE LLM

le chemin inverse : elles caractérisent intrinsèquement la requête (difficulté estimée, embedding sémantique, auto-évaluation par un petit modèle) puis assignent cette caractérisation à une politique de sélection. Des approches hybrides combinent les deux signaux : typiquement les cascades, qui essaient séquentiellement les modèles du moins cher au plus cher avec validation intermédiaire. La présente section parcourt ces familles (§2.3.1 et §2.3.2), puis propose une taxonomie révisée à trois axes (§2.3.3) qui servira de grille de lecture pour le positionnement d’InferRouter-LLM en §2.5.

2.3.1 Approches centrées modèle (model-centric)

Les approches model-centric postulent que le prédicteur de qualité le plus informatif n’est pas la requête elle-même, mais la cartographie relative des LLM candidats sur un espace de compétences. Le routeur exploite alors un signal de préférence agrégé (jugements humains, reward models, signaux contrastifs) pour apprendre, pour chaque requête entrante, une projection vers le modèle attendu comme “le mieux adapté”. Quatre travaux structurent l’état de l’art de cette famille.

2.3.1.0.1 RouteLLM : routage à partir de préférences humaines. L’apport principal de cette famille est le framework ROUTELLM publié par Ong *et al.* à ICLR 2025 [28]. Les auteurs formalisent le routage binaire entre un LLM “fort” et un LLM “faible” comme un problème d’apprentissage à partir de données de préférences issues de Chatbot Arena. Quatre architectures de routeurs y sont comparées : (i) une *Matrix Factorization* sur la matrice (modèle×requête) de scores de victoires ; (ii) un *BERT Classifier* fine-tuné sur les paires d’arena ; (iii) un *Causal LLM Classifier* qui exploite la représentation profonde d’un modèle de fondation ; et (iv) un *Similarity-Weighted ranking* (Similarity-Weighted ranking (SW-ranking)) qui effectue, pour chaque requête entrante q , un calcul d’Elo pondéré par la similarité d’embeddings entre q et les requêtes d’arena pour lesquelles les jugements humains sont disponibles [28]. Plus précisément, en notant E_m le classement Elo du modèle m et $s(q, q_i)$ la similarité cosinus entre l’embedding de q et celui d’une requête d’arena q_i , le score routeur de SW-ranking pour un modèle m s’écrit conceptuellement :

$$R_{\text{SW}}(m, q) = \sum_{i \in \mathcal{A}} s(q, q_i) E_m^{(i)}, \quad (2.1)$$

où $\mathcal{A}$ désigne l’ensemble des requêtes d’arena disponibles et $E_m^{(i)}$ le score Elo de m relativement à cette sous-population de requêtes. SW-ranking constitue ainsi un routeur non-paramétrique qui ne

2.3. ROUTAGE ET SÉLECTION DYNAMIQUE DE LLM

nécessite ni entraînement supervisé dédié ni reward model : seules les préférences d’arena et un modèle d’embedding sont requis [28]. Les auteurs rapportent qu’“avec une calibration sur Chatbot Arena, l’approche réduit le coût d’inférence d’un facteur supérieur à deux sans dégradation de qualité” [28]. Cette explicitation du mécanisme SW-ranking lève l’ambiguïté relevée par le tuteur sur ce terme : SW-ranking désigne spécifiquement le routeur Elo-pondéré par similarité d’embeddings de RouteLLM, et non une catégorie générique de routeur.

2.3.1.0.2 Zooter : distillation de reward model en routeur léger. Lu *et al.* [29] proposent à NAACL 2024 le système ZOOPER, qui aborde la même classe de problème par une voie différente : un reward model généraliste coûteux est invoqué hors-ligne sur un corpus de requêtes d’entraînement pour produire un signal de qualité par couple (requête, modèle) ; ce signal est ensuite distillé en une fonction de routage légère, capable d’opérer en ligne sans invocation du reward model. Une étape de *tag-based label enhancement* atténue le bruit des récompenses brutes. Évalué sur 26 sous-tâches, Zooter surpasse en moyenne le meilleur modèle individuel et remporte la tête sur 44 % des tâches, tout en éliminant le coût d’inférence du reward model [29]. Cette logique “oracle coûteux → routeur léger” est conceptuellement proche du couplage LLM Juge → Estimateur de complexité retenu pour InferRouter-LLM, en particulier dans la phase d’auto-calibration traitée au chapitre 4.

2.3.1.0.3 RouterDC : apprentissage contrastif dual. Toujours du côté model-centric, Chen *et al.* [30] introduisent à NeurIPS 2024 le framework ROUTERDC, qui apprend la projection requête → modèle via deux pertes contrastives conjointes : une perte *sample-LLM* qui rapproche la représentation d’une requête des LLM ayant historiquement bien répondu à des requêtes similaires, et une perte *sample-sample* qui régularise les représentations des requêtes proches sémantiquement. Les auteurs rapportent une amélioration de +2,76 % in-distribution et +1,90 % out-of-distribution par rapport à la baseline ZOOPER [30]. RouterDC montre que l’apprentissage contrastif, par construction adapté aux espaces de représentation continus, est un substrat efficace pour le routage multi-LLM ; cette piste inspire en partie la conception du Complexity Estimator d’InferRouter-LLM (chapitre 4), qui exploite des embeddings SENTENCE-BERT comme représentation intermédiaire.

2.3.1.0.4 Tryage : routeur “perceptif” inspiré du thalamus. Hari et Thomson [31] présentent un travail précurseur publié dès août 2023, qui pousse l’approche model-centric à son extrême : un

2.3. ROUTAGE ET SÉLECTION DYNAMIQUE DE LLM

LLM-routeur dédié prédit la performance attendue de chaque modèle candidat sur la requête entrante, en intégrant simultanément les contraintes utilisateur (empreinte mémoire, fraîcheur, conformité sécurité) au sein d’une fonction objectif unique. Évalué sur plus de 200 000 modèles HuggingFace, TRYAGE atteint une exactitude de sélection de 50,9 %, surpassant GPT-3.5 Turbo et le système Gorilla [31]. La limite structurelle de cette approche est explicitement documentée par les auteurs : le routeur étant lui-même un LLM, son coût d’inférence est non négligeable. Ce constat motive directement le choix, dans InferRouter-LLM, d’un classifieur sklearn léger (< 15 ms) en lieu et place d’un LLM-routeur (cf. chapitre 4).

L’ensemble des approches model-centric partage une hypothèse : la qualité attendue d’un modèle sur une requête dépend principalement de caractéristiques apprises sur le modèle, plutôt que de caractéristiques intrinsèques de la requête. Cette hypothèse est puissante mais comporte une limite : elle requiert un corpus de préférences ou un reward model généraliste pour calibrer la représentation des modèles. La sous-section suivante examine la démarche opposée, qui privilégie la caractérisation intrinsèque de la requête.

2.3.2 Approches centrées requête (query-centric)

Les approches query-centric postulent au contraire que le prédicteur le plus informatif réside dans la requête elle-même : une fois sa difficulté intrinsèque correctement estimée, la politique de sélection devient une simple correspondance entre niveau de difficulté et capacité du modèle cible. Cette famille inclut historiquement les premières formalisations du compromis coût-qualité dans le routage LLM, et fournit l’épine dorsale méthodologique d’InferRouter-LLM.

2.3.2.0.1 FrugalGPT : référence sur le compromis coût-qualité. La contribution la plus citée de cette famille est FRUGALGPT, publiée par Chen *et al.* aux *Transactions on Machine Learning Research* en 2024 [10]. Les auteurs identifient trois leviers de réduction du coût d’inférence : (i) *prompt adaptation* (compression et restructuration du prompt avant invocation) ; (ii) *LLM approximation* (substitution d’un LLM cher par une combinaison cache de réponses + petit modèle de complétion) ; et (iii) *LLM cascade*, principe selon lequel les modèles sont interrogés par ordre croissant de coût, chaque réponse étant validée par un scorer auxiliaire avant escalade éventuelle vers un modèle plus capable. Les auteurs rapportent que “FrugalGPT peut égaler les performances du meilleur LLM

2.3. ROUTAGE ET SÉLECTION DYNAMIQUE DE LLM

individuel (par exemple GPT-4) avec jusqu’à 98 % de réduction de coût, ou améliorer la précision de GPT-4 de +4 % à coût équivalent” [10]. L’amplitude de gain typiquement rapportée sur les benchmarks considérés (HEADLINES, OVERRULING, COQA) se situe entre 50 % et 98 % selon la configuration [10]. FRUGALGPT constitue ainsi la source canonique pour toute affirmation chiffrée relative au gain économique d’un routage multi-modèles, précaution méthodologique explicitement requise dans la version révisée du présent mémoire.

2.3.2.0.2 Hybrid LLM : routage paramétré par un seuil de qualité. Ding *et al.* [32] formalisent à ICLR 2024 une variante *quality-aware* du routage binaire petit/gros modèle. Le routeur, fondé sur un encodeur BERT, prédit pour chaque requête une *difficulty score* définie comme la probabilité que le petit modèle soit aussi bon que le gros sur cette requête ; une politique à seuil ajustable au test-time exploite ce score pour arbitrer. Les auteurs distinguent trois variantes du routeur (déterministe, probabiliste, probabiliste à transformation) et montrent qu’on peut économiser jusqu’à 40 % des invocations du gros modèle sans dégradation de qualité sur le benchmark MixInstruct [32]. Cette formalisation explicite d’une notion de difficulté intrinsèque à la requête, séparée de la notion de coût ou de qualité du modèle, est directement réutilisée dans la définition de l’estimateur de complexité d’InferRouter-LLM (chapitre 3).

2.3.2.0.3 AutoMix : auto-vérification et escalade dynamique. Madaan *et al.* [33] proposent à NeurIPS 2024 le framework AUTOMIX, qui pousse le principe de cascade un cran plus loin : chaque modèle de la cascade est sollicité non seulement pour produire une réponse, mais aussi pour fournir une auto-évaluation few-shot de sa propre fiabilité. Un *meta-verifier* fondé sur un modèle de décision Partially Observable Markov Decision Process (POMDP) arbitre alors entre acceptation de la réponse, escalade vers un modèle plus capable, ou abstention. Les auteurs rapportent une réduction de coût supérieure à 50 % pour une performance comparable [33]. L’intérêt méthodologique d’AutoMix est de démontrer la viabilité d’une auto-évaluation intégrée, piste reprise dans le module LLM Juge d’InferRouter-LLM (chapitre 4) bien que sous une forme distincte (juge externe indépendant plutôt qu’auto-vérification).

2.3.2.0.4 LLM-Blender : ensembling, contrepoint au routage. Jiang *et al.* [34] présentent à ACL 2023 le framework LLM-BLENDER, qui n’est pas à proprement parler un routeur mais un assembleur :

2.3. ROUTAGE ET SÉLECTION DYNAMIQUE DE LLM

pour chaque requête, K modèles candidats sont invoqués en parallèle puis fusionnés par un module *PairRanker* (comparaison pairwise par cross-attention) suivi d'un *GenFuser* (fusion générative). L'approche surpasse tout LLM individuel sur le benchmark MixInstruct créé par les auteurs [34]. LLM-BLENDER constitue un contrepoint instructif au paradigme du routage : en invoquant tous les modèles, l'ensemblage sacrifie l'objectif d'économie de coût au profit de la qualité maximale ; il convient donc mieux aux contextes off-line, mais demeure incompatible avec les contraintes de latence et de budget visées par InferRouter-LLM.

2.3.2.0.5 Eagle : routeur training-free à base d'Elo. Zhao *et al.* [35] présentent à NeurIPS 2024 Workshops le routeur EAGLE, qui combine un Elo global (capacités générales des modèles) avec un Elo local par tâche (capacités spécialisées), sans nécessiter d'entraînement supervisé. Les auteurs rapportent une initialisation $20\times$ plus rapide que les baselines supervisées et un gain d'Area Under the Curve (AUC) de 4 à 23 % sur des benchmarks classiques [35]. EAGLE fournit un argument important : un routeur efficace peut, sous certaines conditions, être construit sans corpus d'entraînement dédié. Ce constat sera mobilisé en §2.5 pour justifier que le recours à l'entraînement supervisé dans InferRouter-LLM se justifie par la spécificité du domaine réseau (calibration sur un dataset d'intents annotés), et non par une nécessité générique du routage.

2.3.2.0.6 Routage à partir de benchmarks publics. Shnitzer *et al.* [36] reformulent à COLM 2024 le problème de sélection d'un LLM comme un problème d'apprentissage entraînable à partir des benchmarks publics existants (MMLU, HellaSwag, etc.). La sélection d'un modèle optimal pour une tâche inconnue est réduite à un ensemble de tâches de classification binaire ("le modèle m va-t-il réussir cette requête ?") apprises sur les scores des benchmarks. Les auteurs montrent une amélioration consistante par rapport à la sélection uniforme du *single-best model*, tout en pointant le risque d'overfitting aux benchmarks [36]. Cette démonstration, à savoir qu'un router performant peut être appris à partir d'un volume modéré de données étiquetées, est essentielle pour InferRouter-LLM, qui s'appuie sur un dataset de quelques centaines d'intents réseau générés par LLM, annotés en domaine, complexité et criticité (chapitre 5).

L'ensemble des approches query-centric converge vers un schéma commun : l'estimation préalable d'une difficulté par requête autorise un routage efficace, et le compromis coût-qualité peut être rendu

2.3. ROUTAGE ET SÉLECTION DYNAMIQUE DE LLM

pilotable par un seuil ajustable. Ce schéma constitue le socle méthodologique d’InferRouter-LLM ; il est cependant insuffisant pour caractériser de manière structurée l’ensemble de la littérature, qui mélange souvent les deux logiques. La sous-section suivante propose une taxonomie à trois axes destinée à y remédier.

2.3.3 Taxonomie révisée

À la suite de l’examen précédent, nous proposons une taxonomie à trois axes destinée à positionner les approches existantes de manière discriminante. Cette grille s’inspire de la formalisation unifiée proposée par Dekoninck *et al.* [37], qui démontre que routing et cascading sont deux instances d’un même problème d’optimisation sous contrainte de coût, en y ajoutant deux dimensions opérationnelles absentes de leur cadre : la granularité de la décision et l’adaptation temporelle. La taxonomie précédemment esquissée dans une version antérieure de ce chapitre mentionnait un troisième axe nommé “Entraînement” ; cette appellation était imprecise et susceptible de prêter à confusion, comme l’a relevé le tuteur. Nous le redéfinissons ci-dessous comme “adaptation dans le temps”, ce qui couvre rigoureusement les approches de cascading hybride et d’apprentissage en ligne présentes dans la littérature, et qui était l’intention initiale de cet axe.

2.3.3.0.1 Axe 1 : Granularité de décision. Cet axe distingue les routeurs opérant par requête (*per-query*), qui prennent une décision indépendante pour chaque entrée, des routeurs opérant par session ou par tâche (*per-session*), qui regroupent plusieurs requêtes sous une même décision de routage. La quasi-totalité des travaux antérieurs adopte la granularité *per-query*, qui maximise la finesse d’arbitrage [28, 32, 30] ; l’approche *per-session* reste l’apanage de systèmes agentiques où un même LLM est sollicité répétitivement dans le contexte d’une même conversation, tels qu’ECOASSISTANT [38] qui exploite la récupération de démonstrations issues de la même session. InferRouter-LLM se positionne sur la granularité *per-query*, cohérente avec le modèle opérationnel d’un IBNS où chaque intent est traité indépendamment.

2.3.3.0.2 Axe 2 : Source d’information. Cet axe partitionne les approches selon la nature du signal exploité par le routeur. Trois catégories se dégagent :

- caractéristiques modèle (*model-centric*) : le signal privilégié est une cartographie apprise des

2.3. ROUTAGE ET SÉLECTION DYNAMIQUE DE LLM

LLM cibles, qu'il s'agisse de préférences humaines [28], de scores d'un reward model distillé [29], d'embeddings appris par contrastive learning [30], ou d'une prédiction de performance multi-objectifs [31] ;

- caractéristiques requête (*query-centric*) : le signal privilégié est intrinsèque à la requête elle-même, sous forme d'un score de difficulté prédit [32, 10, 36] ou d'une variable latente psychométrique à la manière de l'Item Response Theory (IRT), cadre qui estime la difficulté latente d'un item à partir des réponses observées [39] ;
- retour d'exécution (hybride) : le signal est obtenu en invoquant un modèle candidat puis en évaluant sa réponse, soit par auto-vérification [33], soit par un vérificateur externe [10], soit en escaladant la cascade séquentiellement [38, 37].

InferRouter-LLM se classe dans la catégorie *query-centric* : le Complexity Estimator (chapitre 4) opère une caractérisation intrinsèque de l'intent réseau à partir de son embedding sémantique, sans invoquer les LLM candidats au préalable.

2.3.3.0.3 Axe 3 : Adaptation dans le temps. Cet axe distingue les routeurs statiques, dont la politique est figée après entraînement et ne s'ajuste pas en opération, des routeurs adaptatifs, qui modifient leur politique en ligne en fonction des observations cumulées. Deux mécanismes principauxinstancient l'adaptation : (i) la cascade hybride, qui ajuste implicitement la politique en escaladant séquentiellement les modèles jusqu'à obtention d'une réponse satisfaisante, typique de FrugalGPT [10] et formalisée par Dekoninck *et al.* [37] ; (ii) l'apprentissage en ligne, qui exploite la rétroaction opérationnelle (jugements utilisateur, résultats de vérification) pour mettre à jour les paramètres du routeur ou la base de démonstrations utilisées, comme dans ECOASSISTANT [38] et AUTOMIX [33]. La grande majorité des travaux présentés en §2.3.1 relie cette adaptation à une boucle d'évaluation externe ; InferRouter-LLM se positionne sur cet axe en utilisant le LLM Juge local (chapitre 4) comme oracle d'auto-calibration périodique du Complexity Estimator, ce qui s'apparente à l'apprentissage en ligne sans exiger de boucle utilisateur explicite.

Le tableau 2.1 synthétise le positionnement des travaux discutés dans cette section sur les trois axes proposés. Cette grille servira de référence en §2.5 pour caractériser le *gap* méthodologique adressé par InferRouter-LLM, en particulier la combinaison rare de trois traits : granularité per-query,

2.4. ÉVALUATION AUTOMATIQUE DE QUALITÉ (LLM-AS-A-JUDGE)

source query-centric ancrée dans un domaine spécialisé (intents réseau), et adaptation temporelle par auto-calibration via un juge local.

Table 2.1: Positionnement des principaux travaux de routage de LLM selon la taxonomie à trois axes.

Travail	Granularité	Source d’info.	Adaptation
RouteLLM [28]	per-query	model-centric	statique
Zooter [29]	per-query	model-centric	statique
RouterDC [30]	per-query	model-centric	statique
Tryage [31]	per-query	model-centric	statique
FrugalGPT [10]	per-query	query + exécution	cascade
Hybrid LLM [32]	per-query	query-centric	statique
AutoMix [33]	per-query	query + exécution	cascade + en-ligne
LLM-Blender [34]	per-query	exécution (ensembling)	statique
Eagle [35]	per-query	model-centric	statique
Shnitzer <i>et al.</i> [36]	per-query	query-centric	statique
EcoAssistant [38]	per-session	exécution	en-ligne
IRT-Router [39]	per-query	query-centric	statique
InferRouter-LLM (ce travail)	per-query	query-centric (domaine)	en-ligne (juge)

2.4 Évaluation automatique de qualité (LLM-as-a-Judge)

Les sections précédentes ont mis en évidence que tout routeur multi-LLM s’appuie, explicitement ou non, sur une estimation de la qualité attendue de la réponse produite par chaque modèle candidat. Dans la formalisation retenue au chapitre 3, cette qualité attendue est notée $\hat{q}_i(e)$ (cf. éq. (3.4)) ; elle est calibrée par la qualité $q_i(e)$ mesurée a posteriori sur les réponses. Or, l’hypothèse de travail H2 (§ 3.3.5) postule précisément l’absence d’opérateur humain dans la boucle opérationnelle : il faut donc disposer d’une procédure d’évaluation automatique de $q_i(e)$, suffisamment fiable pour piloter une décision de routage et suffisamment légère pour ne pas annihiler le gain économique recherché. Deux familles méthodologiques structurent l’état de l’art : d’une part, les métriques classiques héritées de la génération de langage naturel (BLEU, ROUGE, BERTScore), qui mesurent une proximité textuelle à une référence ; d’autre part, le paradigme LLM-as-a-Judge, qui mobilise un LLM comme juge automatique. Le survey récent de Gu *et al.* [40] cartographie cette seconde famille et en distingue les principaux formats (pointwise, pairwise, listwise) ainsi que les biais structurels associés ; nous l’adoptons comme fil directeur de la présente section. Après avoir explicité les limites des métriques classiques (§ 2.4.1), nous détaillons les contributions structurantes du paradigme LLM-as-a-Judge en concentrant l’analyse sur la méthode ROCKETEVAL, retenue comme méthode de jugement d’InferRouter-LLM

(§ 2.4.2).

2.4.1 Méthodes classiques (BLEU, ROUGE, BERTScore)

Les premières tentatives d'évaluation automatique de la qualité d'une sortie textuelle remontent aux travaux pionniers sur la traduction automatique. Trois métriques structurent historiquement ce paysage et demeurent largement utilisées comme référence comparative.

2.4.1.0.1 BLEU : métrique à base de n-grammes. Proposée par Papineni *et al.* en 2002 pour l'évaluation de la traduction automatique, Bilingual Evaluation Understudy (BLEU) (*Bilingual Evaluation Understudy*) repose sur le calcul de la précision modifiée des n-grammes communs entre une sortie candidate et un ensemble de références, pondérée par une pénalité de brièveté (*brevity penalty*) destinée à pénaliser les sorties artificiellement courtes. Sa simplicité d'implémentation et son interprétabilité immédiate en ont fait pendant deux décennies la métrique de référence en génération de texte. Sa principale faiblesse, abondamment documentée dans la littérature ultérieure, tient à sa nature lexicale-superficielle : deux paraphrases sémantiquement équivalentes mais utilisant des lexiques disjoints obtiennent un score arbitrairement bas, ce qui rend la métrique inadaptée aux tâches autorisant une diversité formelle légitime [41].

2.4.1.0.2 ROUGE : métrique *recall-oriented*. Introduite par Lin en 2004 dans le contexte de l'évaluation des résumés automatiques, Recall-Oriented Understudy for Gisting Evaluation (ROUGE) (*Recall-Oriented Understudy for Gisting Evaluation*) renverse la perspective de BLEU : au lieu de mesurer la précision de la sortie candidate, elle évalue le rappel des n-grammes ou des plus longues sous-séquences communes (LCS) présents dans la référence. Cette orientation rappel la rend particulièrement adaptée aux tâches où la couverture compte davantage que la concision. Comme BLEU, elle souffre toutefois du même verrou structurel : la dépendance à une référence textuelle fixe et l'incapacité à reconnaître les paraphrases sémantiquement valides [41].

2.4.1.0.3 BERTScore : métrique sémantique contextuelle. Zhang *et al.* [41] proposent à ICLR 2020 le basculement méthodologique le plus significatif de cette génération : remplacer les comparaisons lexicales par des comparaisons d'embeddings contextuels produits par un modèle de type BERT.

2.4. ÉVALUATION AUTOMATIQUE DE QUALITÉ (LLM-AS-A-JUDGE)

Chaque *token* de la sortie candidate est apparié à son plus proche voisin dans la référence au sens de la similarité cosinus entre embeddings, puis les scores sont agrégés en précision, rappel et F_1 par moyenne pondérée. Évaluée sur 363 systèmes de traduction et de captioning d’images, BERTScore améliore significativement la corrélation avec les jugements humains par rapport à BLEU et ROUGE, tout en se montrant plus robuste face aux exemples adversariaux du benchmark PAWS [41]. BERTScore demeure cependant tributaire de l’existence d’une référence textuelle et capture mal la cohérence globale d’un texte long.

2.4.1.0.4 Limites partagées pour le cas réseau. Trois limites structurelles, communes à ces trois métriques, les rendent intrinsèquement inadaptées à l’évaluation des sorties produites par un LLM cible sur un intent réseau. Premièrement, le biais lexical : BLEU et ROUGE punissent mécaniquement les paraphrases sémantiquement équivalentes, ce qui est particulièrement pénalisant pour des configurations réseau, où plusieurs séquences syntaxiques distinctes peuvent réaliser le même comportement opérationnel. Deuxièmement, l’exigence d’une référence : les trois métriques requièrent une réponse gold pour calculer le score ; or, pour les intents réseau qui structurent notre cas d’usage (cf. § 2.1.2), il n’existe généralement pas de réponse de référence unique : une même intention opérationnelle peut donner lieu à plusieurs configurations valides, fonctionnellement équivalentes mais lexicalement distinctes. Troisièmement, l’inadaptation aux sorties multi-facettes et long-form : la qualité d’une configuration réseau ne saurait être réduite à un score de proximité textuelle, puisqu’elle engage des dimensions multiples (correction syntaxique, cohérence sémantique, conformité à la politique exprimée, adéquation aux contraintes implicites du domaine) qu’aucune métrique lexicale n’isole. Cette inadéquation, déjà relevée dans le contexte plus large de la génération de configurations [21, 22], motive directement le recours au paradigme LLM-as-a-Judge examiné dans la sous-section suivante.

2.4.2 LLM-as-a-Judge (G-Eval, RocketEval ICLR 2025)

Le paradigme LLM-as-a-Judge désigne l’utilisation d’un LLM lui-même comme évaluateur automatique d’une sortie textuelle produite par un autre LLM (ou par le même). Il émerge à partir de 2023 comme une réponse directe aux limites des métriques classiques : le juge LLM, par sa capacité de raisonnement généraliste, peut intégrer plusieurs dimensions de qualité sans nécessiter de référence figée et accepter les variations paraphrastiques légitimes [40]. Cinq contributions structurent cet état de l’art, dont la

2.4. ÉVALUATION AUTOMATIQUE DE QUALITÉ (LLM-AS-A-JUDGE)

dernière, ROCKETEVAL, est retenue comme méthode de jugement d’InferRouter-LLM.

2.4.2.0.1 G-Eval : premier framework LLM-as-a-Judge à large diffusion. Liu *et al.* [42] présentent à EMNLP 2023 le framework G-EVAL, qui formalise l’usage de GPT-4 comme juge pour l’évaluation de tâches de génération de langage naturel. L’approche articule deux étapes : d’abord, une génération auto-induite d’une chaîne de raisonnement (*Chain-of-Thought*) à partir de l’énoncé de la tâche et des critères d’évaluation ; ensuite, l’attribution d’un score numérique par complétion guidée (*form-filling*). Évalué sur des tâches de résumé automatique, G-EVAL atteint une corrélation de Spearman de 0,514 avec les jugements humains, surpassant significativement les baselines BLEU, ROUGE et BERTScore [42]. Les auteurs identifient toutefois un biais structurel important : la préférence du juge GPT-4 pour les sorties produites par GPT-4 lui-même (*self-preference bias*), qui pose un problème méthodologique lorsqu’on souhaite arbitrer entre plusieurs LLM candidats dont l’un est le juge. La principale limite opérationnelle de G-EVAL reste sa dépendance à un modèle cloud propriétaire onéreux (GPT-4), incompatible avec une évaluation à grande échelle en boucle fermée.

2.4.2.0.2 Zheng *et al.* : évaluer le juge lui-même. Zheng *et al.* [43], dans une contribution publiée à NeurIPS 2023 (*Datasets and Benchmarks Track*), proposent une méta-évaluation systématique du paradigme LLM-as-a-Judge. Les auteurs introduisent deux bancs d’essai complémentaires, MT-BENCH (questions multi-tours difficiles) et *Chatbot Arena* (*battles pairwise issues du crowdsourcing*), et mesurent l’agrément entre les jugements produits par GPT-4 et ceux produits par des annotateurs humains. Le résultat principal est notable : “les juges LLM puissants comme GPT-4 peuvent retrouver les préférences humaines contrôlées et crowdsourcées, avec un agrément supérieur à 80 %, soit le même niveau d’agrément qu’entre annotateurs humains” [43]. Cette validation empirique établit la légitimité méthodologique du paradigme. Les auteurs documentent toutefois quatre catégories de biais récurrents qu’il convient de mitiger : biais de position (l’ordre des candidats influence le jugement), biais de verbosité (préférence pour les réponses longues), biais d’auto-favoritisme (*self-enhancement*) et limitations de raisonnement sur les tâches numériques. Ces biais définissent le cahier des charges implicite des méthodes postérieures, et seront explicitement adressés par les approches checklist (cf. infra).

2.4. ÉVALUATION AUTOMATIQUE DE QUALITÉ (LLM-AS-A-JUDGE)

2.4.2.0.3 JudgeLM : juge spécialisé par fine-tuning. Zhu *et al.* [44] présentent à ICLR 2025 (*Spotlight*) le framework JUDGE_{LM}, qui aborde le problème par une voie différente : plutôt que d’invoquer un LLM généraliste comme juge, les auteurs fine-tunent des modèles open-source (7B, 13B, 33B paramètres) sur un corpus d’annotations produites par un LLM teacher (GPT-4). Trois techniques spécifiques atténuent les biais identifiés par Zheng *et al.* : *swap augmentation* contre le biais de position, *reference support* et *reference drop* contre le biais de connaissance. Le modèle JUDGE_{LM}-7B atteint un agrément supérieur à 90 % avec son juge teacher GPT-4 tout en évaluant 5 000 échantillons en trois minutes sur huit GPU A100 [44], soit un gain de débit considérable face à une invocation API. Cette approche valide empiriquement que la fonction de jugement peut être transférée vers un modèle plus léger, condition nécessaire à un déploiement local. Elle requiert toutefois une phase de fine-tuning dédiée, ce qui en fait une approche moins agile lorsque les critères d’évaluation évoluent.

2.4.2.0.4 PandaLM : juge open-source reproductible. Wang *et al.* [45] proposent à ICLR 2024 le juge PANDA_{LM}, fine-tuné sur LLaMA-7B et LLaMA-70B à partir d’un corpus d’instructions Alpaca annotées par GPT-3.5. PANDA_{LM}-7B atteint des performances comparables à GPT-3.5 et PANDA_{LM}-70B les surpasse [45]. Le modèle mitige le biais de position par une stratégie d’annotation “*Tie*” appliquée aux cas où un swap des candidats modifie le jugement. L’apport spécifique de PANDA_{LM} est de proposer une chaîne d’évaluation entièrement open-source, sans dépendance API, donc compatible avec des contraintes de confidentialité ou de reproductibilité. C’est un argument déterminant dans les contextes opérationnels réseau où les configurations évaluées peuvent contenir de l’information sensible.

Les quatre contributions précédentes (G-EVAL, Zheng *et al.*, JUDGE_{LM}, PANDA_{LM}) tracent une trajectoire claire : validation du paradigme, identification des biais, spécialisation par fine-tuning, ouverture à l’open-source. Toutes partagent toutefois deux limites communes : un format d’évaluation *pointwise* ou *pairwise* difficile à expliquer en termes opérationnels (le juge produit un score numérique ou une préférence, mais pas une justification structurée), et un coût d’inférence qui demeure significatif dès lors que l’on souhaite évaluer des dizaines de milliers de couples (intent, réponse) dans une boucle d’auto-calibration. La méthode ROCKETEVAL, retenue par InferRouter-LLM, répond précisément à cette double limitation.

2.4. ÉVALUATION AUTOMATIQUE DE QUALITÉ (LLM-AS-A-JUDGE)

2.4.2.0.5 RocketEval : évaluation par checklist instanciée. Wei *et al.* [12] présentent à ICLR 2025 le framework ROCKETEVAL, méthode adoptée par InferRouter-LLM pour l'évaluation automatique de qualité. Le principe en est le suivant : transformer l'évaluation forme-libre d'une réponse en une série de questions binaires Y/N (une *checklist*) générées en amont à partir du prompt initial. L'évaluation s'organise en trois étapes : (i) *Checklist Creation*, où un LLM puissant génère, pour chaque prompt, une liste de critères vérifiables spécifiques à l'instance ; (ii) *Checklist Grading*, où un LLM léger note indépendamment chaque item Y/N sur la réponse candidate, sans avoir à comparer plusieurs candidats simultanément ; (iii) *Score Prediction*, qui agrège les jugements binaires en un score final via une régression supervisée [12]. L'apport central de cette architecture est double. D'une part, elle découple la complexité du raisonnement *a priori* (génération de la checklist par un LLM puissant, tâche coûteuse mais rare) de la complexité du jugement en ligne (évaluation Y/N item par item, tâche simple confiée à un LLM léger) ; cette dissociation rend la méthode linéairement scalable et quasi insensible au biais de position. D'autre part, la checklist constitue une trace explicable du jugement, ce qui adresse le verrou d'explicabilité identifié plus haut.

Le résultat empirique central est particulièrement adapté à notre contexte : “ROCKETEVAL, utilisant Gemma-2-2B comme juge, atteint une corrélation de 0,965 avec les préférences humaines, comparable à celle obtenue par GPT-4o, tout en réduisant le coût d'évaluation à grande échelle d'un facteur supérieur à 50” [12]. Cette quasi-équivalence avec GPT-4o sur un modèle de 2 milliards de paramètres exécutable en local change radicalement l'économie d'une évaluation en boucle fermée : la qualité $q_i(e)$ devient mesurable sans coût marginal d'API, ce qui rend tractable l'invocation systématique du juge à chaque itération d'auto-calibration du Complexity Estimator (cf. chapitre 4). C'est cette propriété, une corrélation élevée à GPT-4 pour un modèle exécutable localement, qui justifie le choix de ROCKETEVAL comme socle de la couche d'évaluation d'InferRouter-LLM, instanciée dans notre prototype par `gemma2` servi par Ollama (cf. chapitre 4). Le chapitre 5 confronte ce socle au domaine réseau et en délimite la portée réelle.

En conjuguant des routers adaptatifs (§ 2.3) et des juges automatiques fiables (§ 2.4), une nouvelle génération d'architectures peut envisager un routage dynamique multi-LLM dans des contextes réseau exigeants. La section suivante synthétise ce paysage et positionne InferRouter-LLM par rapport au *gap* méthodologique identifié.

2.5 Synthèse et positionnement

Le parcours mené dans les quatre sections précédentes a successivement établi quatre constats. L’IBN constitue un paradigme conceptuellement mature mais opérationnellement entravé par la carence de traduction lexicale et d’évaluation automatique (§2.1) ; les LLM généralistes appliqués aux tâches réseau lèvent partiellement la première limitation, mais mobilisent quasi-systématiquement un modèle cloud unique sans arbitrage coût-qualité-latence (§2.2) [9, 24, 18] ; la littérature du routage dynamique entre LLM apporte un cadre méthodologique pour cet arbitrage, structuré selon une taxonomie à trois axes (§2.3) [28, 10, 32] ; enfin, le paradigme LLM-as-a-Judge, en particulier ROCKETEVAL [12], débloque l’évaluation automatique de qualité à un coût compatible avec une boucle fermée (§2.4). La présente section synthétise ces quatre éclairages, exhibe le *gap* méthodologique non couvert par la littérature, et positionne InferRouter-LLM comme contribution destinée à le combler.

2.5.1 Comparatif des routers existants

Table 2.2: Comparaison des routers LLM existants face à InferRouter-LLM. Les cellules “N/D” indiquent une information non documentée dans la source primaire.

Router	Type	Domaine	Estim. complexité	Eval. auto. qualité	Cont. cout-lat.
RouteLLM [28]	Query-centric	Generique	SW-ranking / BERT clf.	Win-rate Arena (offline) ¹	✓
FrugalGPT [10]	Hybride	Generique	Cascade sequentielle	Scorer de validation	✓
HybridLLM [32]	Query-centric	Generique	BERT classifier	Difficulty probabiliste	✓
LLM-Blender [34]	Ensembling	Generique	–	PairRanker (pairwise)	–
AutoMix [33]	Hybride	Generique	Auto-verif. few-shot	Meta-verifier POMDP	✓
RouterDC [30]	Query-centric	Generique	Dual contrastive learn.	Implicite (entraînement)	– ²
Manias Semantic [20]	Query-centric	Specialise (5G)	Sentence-BERT / MiniLM	Pre-filtrage sémantique	– ³
InferRouter-LLM (ce travail)	Query-centric	Specialise (reseau)	Sent.-transf. + clf.	LLM-Judge local	✓

2.5. SYNTHÈSE ET POSITIONNEMENT

Le tableau 2.2 récapitule sept travaux représentatifs face à InferRouter-LLM, selon cinq dimensions discriminantes : type de routage, domaine d’application, mécanisme d’estimation de complexité, mécanisme d’évaluation automatique de qualité, et prise en compte des contraintes coût-latence. Quatre lectures verticales permettent d’extraire les régularités structurelles de l’état de l’art.

2.5.1.0.1 Lecture par type de routage. La majorité des travaux retenus se positionnent soit comme query-centric purs, soit comme hybrides. Parmi les premiers, ROUTELLM projette la requête sur un référentiel de préférences humaines via SW-ranking [28], HYBRIDLLM prédit un *difficulty score* via un classifieur BERT [32], et ROUTERDC apprend la projection par perte contrastive duale [30]. Les hybrides combinent une caractérisation de requête et un retour d’exécution, tels que FRUGALGPT et sa *LLM cascade* [10] ou AUTOMIX et son *meta-verifier* POMDP [33]. Les approches strictement model-centric purs apparaissent rares parmi les sept travaux retenus pour le comparatif resserré ; elles dominent en revanche la taxonomie élargie présentée au tableau 2.1. Aucun de ces routers ne combine systématiquement les deux signaux (caractérisation intrinsèque et retour de juge externe), comme le fait InferRouter-LLM en couplant le Complexity Estimator query-centric et le LLM-Juge local d’évaluation post-hoc.

2.5.1.0.2 Lecture par domaine. Le constat est ici saisissant : six des sept travaux comparatifs visent un domaine générique, c’est-à-dire qu’ils ont été développés et validés sur des benchmarks de questions ouvertes (Chatbot Arena, MMLU, MixInstruct) sans ancrage systèmes ou réseaux [28, 10, 32, 34, 33, 30]. Seul SEMANTIC ROUTER de Manias *et al.* [20] se positionne sur un domaine réseau (gestion intent-driven 5G), mais avec une différence structurante : chez ces auteurs, le routage sémantique sélectionne le composant réseau cible (fonction du cœur 5G) vers lequel adresser une requête, et non le LLM cible le mieux adapté à la complexité de l’intent. Aucun travail existant ne combine donc simultanément la spécialisation au domaine réseau et le routage inter-LLM ; cette intersection vide constitue le premier verrou que InferRouter-LLM se propose de lever.

2.5.1.0.3 Lecture par mécanisme d’évaluation automatique. La colonne “Éval. auto. qualité” du tableau 2.2 révèle une hétérogénéité marquée, mais aussi une lacune systématique. ROUTELLM utilise les win-rates Chatbot Arena, signal hors-ligne qui ne peut pas être interrogé au moment du routage [28] ; FRUGALGPT mobilise un scorer de validation à chaque étage de cascade [10] ; HYBRIDLLM se contente

d'un score de difficulté probabiliste appris hors-ligne par le BERT classifier [32] ; AUTOMix introduit un *meta-verifier* POMDP propre à sa cascade [33] ; ROUTERDC encode l'évaluation implicitement dans son objectif contrastif [30]. Aucun de ces dispositifs ne mesure la qualité réelle de la réponse a posteriori dans la boucle de production : tous s'appuient sur des proxys (préférences hors-ligne, score appris, gating de confiance) qui restent corrélés à la qualité mais ne la mesurent pas à l'instance. InferRouter-LLM est, parmi les travaux recensés, le seul à intégrer un LLM-Juge local invocable a posteriori selon la méthode ROCKETEVAL [12], ce qui permet une évaluation factuelle de $q_i(e)$ sans coût marginal d'API.

2.5.1.0.4 Lecture par contraintes coût-latence. Trois travaux seulement formalisent explicitement un compromis coût-qualité : FRUGALGPT via la cascade séquentielle sous budget de coût [10], HYBRIDLLM via un seuil de routage réglable [32], et AUTOMix via la décision POMDP à chaque étage de cascade [33]. ROUTELLM expose un seuil de routage [28], mais ne le couple pas à une borne de latence. Aucun travail recensé n'intègre simultanément les deux contraintes coût et latence avec des bornes dures par requête, ce qui sera précisément formalisé dans le présent mémoire par les contraintes $c_i \leq C_{\max}(e)$ et $\ell_i(e) \leq L_{\max}(e)$ de l'équation (3.5) (cf. §3.3). Cette absence est d'autant plus problématique dans le contexte réseau, où les SLA opérationnels imposent par construction des bornes dures par intent, incompatibles avec une optimisation en moyenne.

2.5.2 Gap identifié et justification d'InferRouter-LLM

L'analyse comparative précédente fait émerger un triple verrou non résolu par l'état de l'art, qui motive directement le présent mémoire. Cette sous-section articule successivement le contenu du verrou, sa pertinence opérationnelle pour les opérateurs réseau, et la manière dont les trois contributions d'InferRouter-LLM y répondent.

2.5.2.0.1 Le triple verrou non résolu. Aucun travail existant ne réunit simultanément trois propriétés dont la conjonction définit le périmètre d'InferRouter-LLM. La lecture par domaine du §2.5.1 a montré que les routers recensés sont soit génériques [28, 10, 32, 34, 33, 30], soit orientés vers des composants réseau plutôt que vers des LLM spécialisés, comme SEMANTIC ROUTER [20] : la spécialisation au domaine réseau manque donc. La lecture par contraintes a établi qu'aucun travail ne combine bornes de

2.5. SYNTHÈSE ET POSITIONNEMENT

coût et bornes de latence avec une granularité *per-intent* : le routage tri-critère sous contraintes dures manque aussi. La lecture par mécanisme d'évaluation a révélé qu'aucun router n'intègre d'évaluation factuelle a posteriori de la réponse produite : l'évaluation automatique fiable et économique manque enfin. Ces trois absences forment le verrou que les contributions ci-dessous adressent une à une.

2.5.2.0.2 Pertinence opérationnelle pour les opérateurs réseau. La conjonction de ces trois propriétés n'est pas un raffinement académique : elle répond à trois exigences opérationnelles indépendamment documentées par la littérature réseau et télécom.

D'abord, les réseaux 5G et au-delà génèrent des intents à la cadence opérateur, plusieurs centaines par jour dans les scénarios d'orchestration de slices décrits par Zeydan et Turk [15] et par Mehmood *et al.* [8]. Cette cadence exclut par construction toute boucle humaine d'évaluation : la phase d'*intent assurance* [7] doit être automatisée, ce qui suppose un juge automatique fiable et économique, propriété exactement adressée par ROCKETEVAL [12].

Ensuite, les LLM cloud propriétaires de classe GPT-4 et Claude restent trop coûteux à l'échelle des opérations réseau lorsque les requêtes se chiffrent en centaines de milliers par mois, comme Zhou *et al.* [18] le soulignent explicitement. Par ailleurs, leur charge système n'est ni accessible ni stable du côté client, conformément à l'hypothèse H3 que nous formulons au chapitre 3 (cf. §3.3.5). Une architecture qui combine routage local pour les intents simples et cloud pour les intents complexes, avec auto-calibration périodique par juge local, est donc strictement plus défendable qu'une invocation cloud uniforme.

Enfin, le contexte réseau impose des SLA stricts par requête : la latence d'activation d'un intent slice 5G uRLLC, par exemple, est bornée par les spécifications 3GPP [8], et le budget opérationnel par requête est borné par les contraintes économiques de l'opérateur. Cette double borne dure rend les routers généralistes optimisant en moyenne sans contrainte explicite [28, 10, 30] structurellement inadaptés au cas réseau ; la formalisation proposée au chapitre 3 adresse précisément cette inadéquation.

2.5.2.0.3 Les trois contributions d'InferRouter-LLM. Aux trois facettes du verrou identifié correspondent trois contributions du présent mémoire, dont chacune sera détaillée au chapitre 4.

2.5.2.0.4 Contribution C1 : Estimateur de complexité sémantique d'intents réseau. Un classifieur léger estime la complexité d'un intent en moins de 15 ms par requête sur un MacBook Air M5. L'estimateur s'appuie sur des attributs calculés $(n, p, |\delta|)$, soit le nombre d'entités, la profondeur d'inférence et le nombre de domaines croisés : des grandeurs lisibles par un opérateur et insensibles à la longueur de l'énoncé, un confond documenté au chapitre 5. L'évaluation y mesure aussi des embeddings de phrase, qui portent un signal de complexité fort à l'échelle des 252 intents et ouvrent une voie combinée. Il est entraîné sur un dataset d'intents réseau généré par LLM, annoté par expert réseau selon les quatre domaines de la taxonomie §2.1.2. Cette contribution adresse le premier verrou (spécialisation au domaine réseau).

2.5.2.0.5 Contribution C2 : Routage tri-critère sous contraintes dures. Le routeur $R^*(e)$ sélectionne, pour chaque intent, le modèle de plus haute qualité attendue $\hat{q}_i(e)$ sur l'ensemble admissible $M(e)$ défini par les bornes $C_{\max}(e)$ et $L_{\max}(e)$ (cf. éq. (3.5) et (3.6)). Ce routage tri-critère est, à notre connaissance, la première formalisation conjointe coût, latence et qualité avec bornes dures *per-intent* dans la littérature du routage de LLM ; il adresse le deuxième verrou.

2.5.2.0.6 Contribution C3 : Évaluation automatique par LLM-Juge local. Un juge local instancié par **gemma2** (2 puis 9 milliards de paramètres) servi via Ollama applique la méthode ROCKETEVAL [12], dont les auteurs rapportent une corrélation de 0,965 avec les préférences humaines pour un coût réduit d'un facteur supérieur à 50 par rapport à un juge cloud [12]. Cette évaluation alimente la calibration des seuils du routeur (cf. axe 3 de la taxonomie en §2.3.3). Notre évaluation (chapitre 5) en précise le périmètre : le juge local départage de manière fiable des candidats nettement différents, ce que le routage demande, mais ne fait pas office d'oracle d'une qualité absolue fine. Cette contribution adresse le troisième verrou pour cet usage.

Le chapitre suivant formalise rigoureusement ce problème de routage d'intents réseau : il pose les ensembles, fonctions et contraintes du modèle, énonce les hypothèses de travail (notamment H3 sur l'inobservabilité de la charge cloud), et fixe les notations qui structurent les trois contributions ci-dessus.

Chapter 3

Définition du problème

Contents

3.1	Modélisation (architecture) système	37
3.1.1	Le routeur comme composant central	38
3.1.2	Pool de LLM génériques et spécialisés	38
3.1.3	LLM Juge local	38
3.1.4	Déploiement edge et co-location	40
3.1.5	Comportement attendu par slice réseau	41
3.2	Challenges du système	42
3.2.1	Nombre d'entités impliquées	42
3.2.2	Profondeur d'inférence requise	43
3.2.3	Domaines croisés	43
3.2.4	Opérationnalisation	44
3.3	Formulation du problème	44
3.3.1	Modélisation d'un intent réseau	44
3.3.2	Pool de modèles cibles	45
3.3.3	Critères d'admissibilité	46
3.3.4	Objectif du routeur	46
3.3.5	Hypothèses opérationnelles	47
3.3.6	Qualité agrégée (AIQ)	48
3.3.7	Latence bout en bout (P50, P99)	49
3.3.8	Coût agrégé proxy	49
3.3.9	Distribution de routage	49
3.3.10	Critère principal de comparaison	50

La Table 3.1 regroupe les notations du cadre formel utilisées dans ce chapitre et les suivants. Les sections 3.1 et 3.2 mobilisent certains symboles avant leur définition formelle (§3.3) ; le tableau donne le renvoi correspondant, sans se substituer aux définitions en prose.

Table 3.1: Notations du cadre formel d’InferRouter-LLM.

Symbole	Signification	Définition
e	intent réseau exprimé en langage naturel	éq. (3.1)
$d(e)$	domaine fonctionnel de l’intent	éq. (3.1)
$\mathcal{D}$	ensemble des domaines fonctionnels	éq. (3.2)
$\kappa(e)$	criticité opérationnelle (low/med/high)	éq. (3.1)
$L_{\max}(e)$	latence maximale tolérée pour e (ms)	éq. (3.1)
$C_{\max}(e)$	coût maximal admissible par inférence	éq. (3.1)
$n(e)$	nombre d’entités réseau distinctes de l’énoncé	§3.2.1
$p(e)$	profondeur d’inférence requise	§3.2.2
$\delta(e)$	sous-ensemble des domaines touchés par e	§3.2.3
M	pool de LLM cibles $\{m_1, \dots, m_k\}$	éq. (3.3)
$M(e)$	sous-ensemble des modèles admissibles pour e	éq. (3.5)
$R^*(e)$	modèle sélectionné par le routeur	éq. (3.6)
$\hat{q}_i(e)$	estimation a priori de la qualité de m_i sur e	éq. (3.4)
$q_i(e)$	qualité mesurée a posteriori par le LLM Juge	§3.3.2
c_i	coût unitaire d’inférence de m_i (proxy tarifaire)	éq. (3.4)
$\ell_i(e)$	latence d’inférence de m_i sur e (ms)	éq. (3.4)
$T(e)$	temps total de traitement de e ($T_{\text{est}} + T_{\text{rout}} + T_{\text{inf}} + T_{\text{juge}}$)	éq. (3.8)
AIQ	qualité moyenne d’inférence sur le jeu d’évaluation	éq. (3.7)

3.1 Modélisation (architecture) système

Avant toute formalisation, nous décrivons l’architecture cible du système et le rôle de chacun de ses composants. InferRouter-LLM reçoit des intents réseau en langage naturel et les achemine vers un modèle de langage adapté, sous contrainte de qualité, de latence et de coût. Le système s’articule autour d’un routeur central, d’un pool de LLM cibles, d’un LLM Juge local et d’un estimateur de complexité. L’ensemble est pensé pour un déploiement edge, au plus près du contrôleur réseau.

3.1.1 Le routeur comme composant central

Le routeur InferRouter-LLM est l'organe de décision du système. Il prend en entrée un intent textuel émis par un opérateur ou un contrôleur IBN, puis choisit le LLM cible vers lequel déléguer l'inférence. Sa décision combine trois signaux. Un estimateur de complexité lui fournit des caractéristiques sémantiques de l'intent. Le pool de modèles lui expose des attributs de coût et de latence par candidat. Un LLM Juge local lui retourne une qualité mesurée *a posteriori*, qui alimente la calibration au fil de l'eau.

Le routeur n'exécute aucune inférence générative lui-même. Il orchestre. Cette séparation isole la logique de décision des modèles sous-jacents et autorise l'ajout ou le retrait d'un LLM cible sans toucher au cœur du routage.

3.1.2 Pool de LLM génériques et spécialisés

Le pool de modèles cibles mêle deux familles. Les LLM génériques (par exemple un modèle propriétaire de grande taille accessible via OpenRouter) couvrent l'ensemble des domaines et servent de repli quand l'intent croise plusieurs champs fonctionnels. Les LLM spécialisés ciblent chacun un domaine réseau : accès radio (RAN), cœur de réseau, sécurité, gestion de slices.

Nous employons uniformément le terme LLM spécialisé pour désigner un modèle du pool affecté à un domaine ; il n'implique aucun comportement agentique (pas d'appel d'outils ni d'action autonome). Un LLM spécialisé est attendu plus précis sur son domaine, à coût souvent inférieur à celui d'un grand modèle généraliste.

Nous retenons cette dualité génériques/spécialisés parce qu'elle matérialise le compromis central du routage. Un intent simple et monodomaine n'a pas besoin d'un grand modèle coûteux. Un intent multi-domaines ne peut pas être confié à un seul LLM spécialisé. Le routeur arbitre entre les deux selon la complexité estimée. La Figure 3.1 résume cette organisation : le routeur central, les deux familles de modèles cibles, et les deux composants en boucle (Juge et estimateur de complexité).

3.1.3 LLM Juge local

L'évaluation de la qualité d'une réponse repose sur un LLM Juge exécuté localement, servi par Ollama (`gemma2`, en 2 puis 9 milliards de paramètres). Le Juge applique la méthode RocketEval [12] : une

3.1. MODÉLISATION (ARCHITECTURE) SYSTÈME

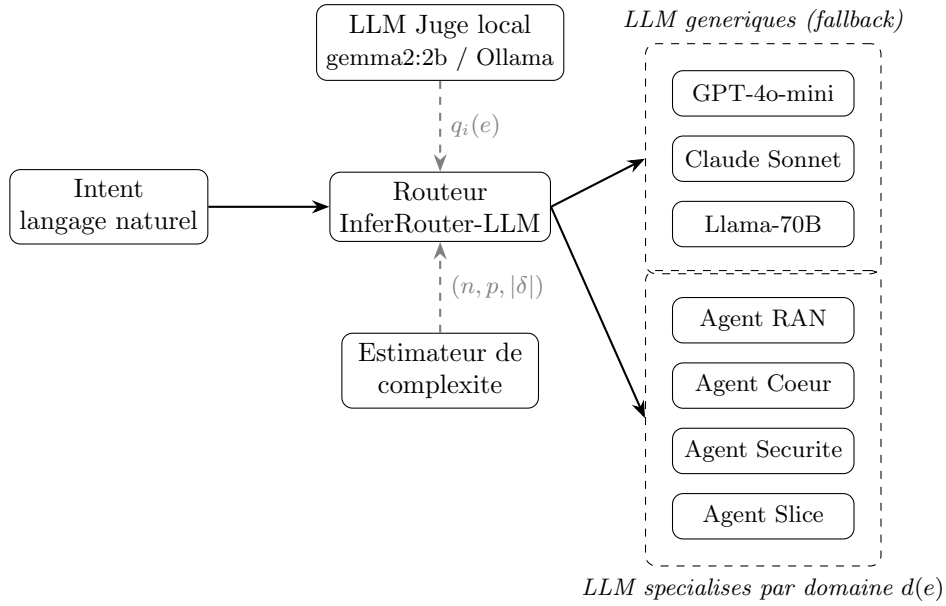


Figure 3.1: Architecture du pool de LLM cibles d'InferRouter-LLM. Le routeur sélectionne entre des LLM génériques (fallback multi-domaine) et des LLM spécialisés par domaine fonctionnel (RAN, cœur, sécurité, slice). Le LLM Juge local fournit la qualité estimée $q_i(e)$ a posteriori ; l'estimateur de complexité fournit les attributs $(n(e), p(e), |\delta(e)|)$.

checklist générée par intent, gradée item par item, agrégée en un score scalaire dans $[0, 1]$. Nous retenons une exécution locale plutôt qu'un juge cloud pour deux raisons. La latence d'évaluation reste maîtrisée et constante. Les intents réseau, potentiellement sensibles, ne quittent pas le périmètre de l'opérateur.

Le rôle exact du Juge est nuancé par l'évaluation du chapitre 5. Nous distinguons deux régimes d'évaluation. La discrimination grossière départage deux candidats nettement différents, par exemple une réponse correcte face à une réponse manifestement fautive. La discrimination fine détecte une erreur isolée dans une réponse par ailleurs correcte. Le juge local est fiable dans le premier régime, pas dans le second.

Le Juge sert donc de signal de routage qui départage les modèles admissibles, et alimente la calibration des seuils hors ligne. Le détail de cet usage, et ses limites, sont établis au chapitre 5 (§5.3).

3.1.4 Déploiement edge et co-location

Le routeur et le LLM Juge sont co-localisés avec le contrôleur IBN, sur un nœud de bordure proche de l'infrastructure gérée. Ce choix de déploiement a une conséquence directe sur le modèle de latence : le temps de transport réseau entre le routeur et le contrôleur reste négligeable devant le temps d'inférence des LLM cibles, eux-mêmes souvent appelés en cloud. Nous y revenons à la §3.3.7.

Les LLM cibles, eux, ne sont pas nécessairement locaux. La plupart sont appelés via OpenRouter, donc en cloud propriétaire. La charge Graphics Processing Unit (GPU) sous-jacente n'est ni observable ni contrôlable depuis le routeur. La décision s'appuie donc sur des grandeurs côté client : latence mesurée, tarif unitaire, qualité estimée. La Figure 3.2 détaille les étapes successives du traitement d'un intent et les composantes de latence associées à chacune.

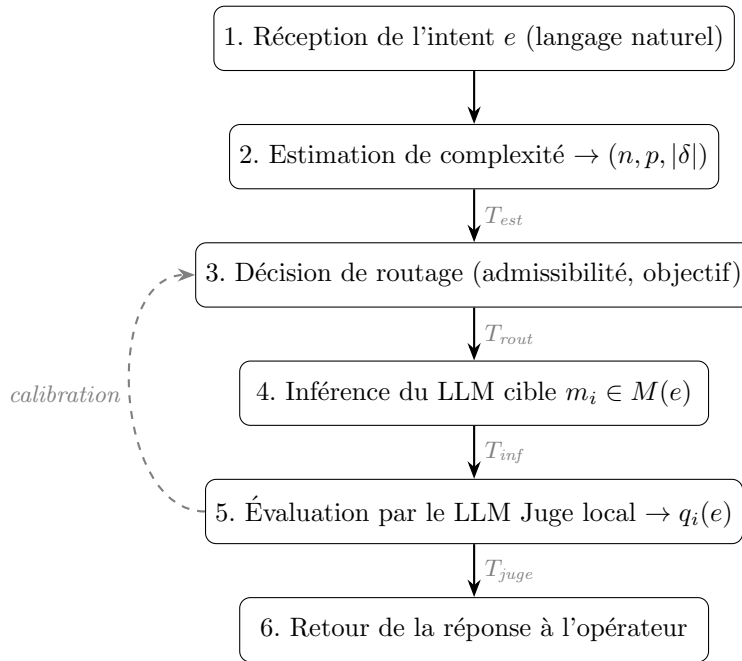


Figure 3.2: Flux de traitement d'un intent par InferRouter-LLM. Les annotations T_{est} , T_{rout} , T_{inf} et T_{juge} matérialisent les composantes de la décomposition de latence (Equation 3.8). La latence de transport réseau est absorbée par la co-localisation du routeur en bordure de l'infrastructure supervisée.

3.1.5 Comportement attendu par slice réseau

Le routage ne peut pas traiter tous les intents de la même façon. Un intent porte sur une slice, et chaque type de slice impose son propre régime de qualité de service. La spécification 3GPP TS 22.261 [46] définit trois familles standard, dont découlent des contraintes opérationnelles distinctes. Ces contraintes se répercutent directement sur les attributs de criticité $\kappa(e)$ et de budget de latence $L_{\max}(e)$ de l'intent, tels que modélisés à l'équation (3.1) (§3.3.1).

L'eMBB (*enhanced Mobile BroadBand*) vise le haut débit pour des usages grand public. La latence radio y reste tolérante, de l'ordre de la centaine de millisecondes, et la criticité opérationnelle est moyenne. Les intents typiques portent sur la configuration de bande passante ou la lecture de KPIs de débit. Pour ces intents, $\kappa(e)$ vaut **med** et $L_{\max}(e)$ peut atteindre l'ordre de la seconde. La latence d'inférence d'un LLM cible reste donc admissible, et un modèle générique de taille moyenne suffit.

L'URLLC (*Ultra-Reliable Low-Latency Communications*) cible la latence très basse, de 1 à 10 ms côté radio, avec une fiabilité de 99,999 %. La criticité y est haute : une reconfiguration de slice ou une réponse à un incident engage la continuité d'un service critique. Pour ces intents, $\kappa(e)$ vaut **high** et le budget $L_{\max}(e)$ se resserre, ce qui restreint le sous-ensemble admissible $M(e)$ (éq. (3.5)) aux modèles les plus rapides. L'admissibilité devient stricte sur la qualité, et un LLM spécialisé RAN s'impose plutôt qu'un généraliste.

Le mMTC (*massive Machine-Type Communications*) connecte un grand nombre de terminaux à faible débit, avec une latence tolérante. Les intents portent sur le provisioning à grande échelle ou les KPIs de connectivité Internet of Things (IoT). La criticité unitaire reste modérée, mais le volume d'intents est élevé : le coût agrégé devient le critère dominant. Le routeur privilégie alors un LLM léger, dont le tarif unitaire c_i faible reste compatible avec un $C_{\max}(e)$ serré à l'échelle de la flotte.

Cette différenciation par slice est une exigence opérationnelle documentée dans la littérature IBN [8]. Elle interdit un seuil de routage unique et justifie que $\kappa(e)$ et $L_{\max}(e)$ dépendent du type de slice visé par l'intent. La Table 3.2 résume cette dépendance. La criticité $\kappa(e)$ n'apparaît pas directement dans les équations de routage de la §3.3 : son effet transite par les bornes $L_{\max}(e)$ et $C_{\max}(e)$, qu'elle conditionne.

3.2. CHALLENGES DU SYSTÈME

Table 3.2: Répercussion du type de slice sur la criticité $\kappa(e)$ et les bornes de l'intent (régimes qualitatifs dérivés des exigences de service de la spécification 3GPP TS 22.261 [46]).

Famille de slice	$\kappa(e)$	Régime de $L_{\max}(e)$	Borne dominante
eMBB	med	tolérant (ordre de la seconde)	aucune (arbitrage libre)
URLLC	high	resserré	$L_{\max}(e)$
mMTC	med	tolérant	$C_{\max}(e)$

3.2 Challenges du système

Le routage d'intents réseau vers un pool de LLM soulève des verrous techniques absents du routage généraliste. La complexité d'un intent est difficile à mesurer *a priori*, avant toute inférence. Aucun oracle humain n'est disponible au runtime pour trancher la qualité d'une réponse. Les contraintes opérationnelles, enfin, varient d'une slice à l'autre, ce qui interdit un seuil unique. Nous détaillons ici la première de ces difficultés : caractériser la complexité d'un intent par des grandeurs observables.

L'estimateur de complexité introduit au chapitre 4 doit opérationnaliser une notion de difficulté propre aux intents réseau, distincte de la difficulté "académique" (raisonnement mathématique, compréhension de texte) explorée par les routeurs généralistes [39, 32]. Trois critères complémentaires, observables par analyse syntaxique et sémantique du libellé textuel, serviront d'entrées au classifieur de complexité.

3.2.1 Nombre d'entités impliquées

Soit $n(e)$ le nombre d'entités réseau distinctes mentionnées dans le libellé d'un intent e , où une entité désigne une slice, une fonction réseau (NF), un indicateur de performance (KPI) ou une politique de sécurité nommée. À titre d'exemple, un intent "quelle est la latence de la slice URLLC #1 ?" implique $n(e) = 2$ (une slice et un KPI de latence), tandis qu'un intent "reconfigurer les deux slices d'un même tenant pour respecter une cible de SLA donnée sans violer une politique de sécurité nommée" atteint $n(e) \geq 5$. Le nombre d'entités constitue un indicateur direct de la surface de raisonnement à couvrir par le modèle cible, suivant la logique décrite par Ding et al. [32] pour leur "difficulty score", et peut être quantifié par un module de reconnaissance d'entités nommées (NER) spécialisé sur le vocabulaire réseau.

3.2.2 Profondeur d'inférence requise

Soit $p(e) \in \mathbb{N}^*$ la profondeur d'inférence d'un intent, définie comme le nombre d'étapes de raisonnement élémentaires nécessaires pour produire la réponse attendue. Un intent direct du type “donne-moi la latence courante du slice S_1 ” correspond à $p(e) = 1$ (une lecture d'état), alors qu'un intent du type “optimise la slice S_1 tout en respectant le SLA σ et en minimisant l'impact sur la slice S_2 ” combine au moins trois étapes (diagnostic de l'état courant, calcul d'un plan d'optimisation, vérification de non-régression), soit $p(e) \geq 3$. Cette dimension de difficulté est intrinsèque à la trame de raisonnement attendue et fait écho à la variable latente de difficulté estimée par le modèle IRT de Song et al. [39], transposée ici au cas réseau.

La profondeur $p(e)$ n'est pas observable directement sur l'énoncé : nous l'approchons par le nombre de contraintes qu'il exprime. Nous appelons contrainte toute condition qui restreint ou oriente la réponse attendue. Deux formes sont comptées : les marqueurs de subordination ou d'objectif (“tout en respectant”, “sans dégrader”, “minimiser”) et les seuils chiffrés associés à un comparateur (“latence inférieure à 10 ms”).

L'intent “optimiser la slice S_1 tout en respectant le SLA σ , sans dégrader la slice S_2 et avec une latence inférieure à 10 ms” compte ainsi trois contraintes : le respect du SLA, la protection de S_2 et le seuil chiffré. Ce comptage, opéré par expressions régulières sur l'énoncé, définit l'attribut `n_constraints` exploité par l'estimateur des chapitres 4 et 5.

3.2.3 Domaines croisés

Soit $\delta(e) \subseteq \mathcal{D}$ le sous-ensemble des domaines fonctionnels concernés par l'intent e , calculé par projection sémantique de l'énoncé sur les quatre catégories de l'équation (3.2). Lorsque $|\delta(e)| = 1$, l'intent est monodomaine et peut être adressé au LLM spécialisé du domaine $d(e)$. Lorsque $|\delta(e)| \geq 2$ (typiquement $\text{RAN} \cap \text{sécurité}$, ou $\text{slice} \cap \text{cœur}$), l'intent est multi-domaine et requiert soit un LLM généraliste capable de couvrir l'union des champs, soit une orchestration explicite entre LLM spécialisés (perspective discutée au chapitre 6). La littérature de routage multi-modèle [37] n'aborde pas explicitement cette dimension, qui constitue une spécificité du contexte intents réseau.

3.3. FORMULATION DU PROBLÈME

3.2.4 Opérationnalisation

Les trois critères $(n(e), p(e), |\delta(e)|)$ constituent les variables explicatives observables de la complexité d'un intent. Leur agrégation en un score scalaire ou vectoriel exploitable par le routeur est confiée à un classifieur léger décrit au chapitre 4 (**sentence-transformers** suivi d'un classifieur **scikit-learn**), inspiré de la démarche IRT-Router [39] mais adapté à la taxonomie réseau de la §3.3.1. La chaîne d'inférence vise une cible de conception inférieure à 15 ms par estimation, valeur fixée pour rester négligeable devant le budget total $L_{\max}(e)$ d'un intent réseau (typiquement de l'ordre de la seconde) et vérifiée expérimentalement au chapitre 5.

3.3 Formulation du problème

Nous posons ici le cadre mathématique du routage. Nous définissons successivement la structure d'un intent réseau, le pool de LLM cibles, les critères d'admissibilité et l'objectif du routeur. Nous énonçons ensuite les hypothèses opérationnelles, puis les métriques de succès. La formulation s'inspire des cadres canoniques de routage qualité/coût [32, 37] et de la projection coût-qualité proposée par RouterBench [47], en y ajoutant les contraintes propres au contexte réseau (criticité opérationnelle, latence dure, domaine fonctionnel).

3.3.1 Modélisation d'un intent réseau

Soit E l'ensemble des intents réseau exprimés en langage naturel par un opérateur. Un intent $e \in E$ désigne une requête de configuration, d'optimisation ou de diagnostic portant sur une infrastructure réseau (par exemple, "créer une slice URLLC à faible latence pour véhicules connectés"). Chaque intent est caractérisé par le quadruplet d'attributs :

$$e \equiv (d(e), \kappa(e), L_{\max}(e), C_{\max}(e)), \quad (3.1)$$

où :

- $d(e) \in \mathcal{D}$ désigne le domaine fonctionnel de l'intent, avec

$$\mathcal{D} = \{\text{RAN, cœur, sécurité, slice}\}; \quad (3.2)$$

3.3. FORMULATION DU PROBLÈME

- $\kappa(e) \in \{\text{low}, \text{med}, \text{high}\}$ représente le niveau de criticité opérationnelle de l'intent, reflétant l'impact potentiel d'une réponse erronée sur le service réseau ; son effet sur les bornes $L_{\max}(e)$ et $C_{\max}(e)$ est résumé à la Table 3.2 ;
- $L_{\max}(e) \in \mathbb{R}_{>0}$ dénote la latence maximale tolérée pour traiter l'intent e , exprimée en millisecondes ;
- $C_{\max}(e) \in \mathbb{R}_{>0}$ correspond au coût maximal admissible par inférence, exprimé comme un proxy adimensionnel dont la définition opérationnelle est donnée en §3.3.6.

3.3.2 Pool de modèles cibles

Le système dispose d'un pool fini de LLM spécialisés par domaine, noté

$$M = \{m_1, m_2, \dots, m_k\}, \quad k \in \mathbb{N}^*, \quad (3.3)$$

dont la composition concrète sera précisée au chapitre 4. À chaque modèle $m_i \in M$ et chaque intent $e \in E$, on associe le triplet caractéristique :

$$m_i \longleftrightarrow (\hat{q}_i(e), c_i, \ell_i(e)), \quad (3.4)$$

où :

- $\hat{q}_i(e) \in [0, 1]$ désigne l'estimation a priori de la qualité de la réponse du modèle m_i pour l'intent e , disponible avant toute inférence. Elle est initialisée puis mise à jour par la calibration hors ligne décrite au chapitre 4 ;
- $c_i \in \mathbb{R}_{>0}$ représente le coût unitaire d'inférence du modèle m_i , supposé indépendant de l'intent (proxy basé sur le tarif par jeton, cf. §3.3.6) ;
- $\ell_i(e) \in \mathbb{R}_{>0}$ dénote la latence d'inférence observée pour le modèle m_i sur l'intent e , en millisecondes.

La qualité effective de la réponse, notée $q_i(e) \in [0, 1]$, est mesurée a posteriori par le LLM Juge local (méthode RocketEval, checklist générée par intent puis gradée, chapitre 4). Le routeur ne l'observe

3.3. FORMULATION DU PROBLÈME

jamais au moment de la décision : elle sert de référence pour calibrer $\hat{q}_i(e)$ et pour évaluer le système au chapitre 5, où elle est exploitée en comparaison relative entre candidats. Cette distinction entre les deux notations lève la circularité apparente entre décision et évaluation : la décision repose sur $\hat{q}_i(e)$, la mesure $q_i(e)$ n'intervient qu'après inférence.

Contrairement à la formulation de Dekoninck et al. [37] qui intègre uniquement une contrainte de coût, nous explicitons une contrainte de latence par intent, nécessaire dans un contexte opérationnel réseau où les SLA imposent des bornes dures.

3.3.3 Critères d'admissibilité

Pour un intent e donné, on définit le sous-ensemble admissible des modèles satisfaisant simultanément les contraintes de latence et de coût :

$$M(e) = \{ m_i \in M : \ell_i(e) \leq L_{\max}(e) \wedge c_i \leq C_{\max}(e) \}. \quad (3.5)$$

L'équation (3.5) définit une enveloppe faisable similaire à celle décrite par RouterBench [47] dans le plan coût-qualité, en y ajoutant la dimension temporelle. Le filtrage par domaine fonctionnel $d(e)$ peut, selon l'implémentation, intervenir en amont (restriction du pool M aux LLM spécialisés du domaine $d(e)$) ou en aval via une pénalité dans $\hat{q}_i(e)$; ce point est traité en §3.2.

3.3.4 Objectif du routeur

Le routeur, formellement une application $R : E \rightarrow M \cup \{\perp\}$, sélectionne pour chaque intent $e \in E$ le modèle admissible de plus haute qualité attendue :

$$R^*(e) = \arg \max_{m_i \in M(e)} \hat{q}_i(e). \quad (3.6)$$

Cette formulation suit la logique d'optimisation sous contraintes adoptée par Dekoninck et al. [37], en remplaçant l'espérance de coût globale par des bornes dures intent-par-intent (équ. (3.5)). La sélection repose sur l'estimation a priori $\hat{q}_i(e)$, elle-même calibrée hors ligne par les mesures a posteriori $q_i(e)$ du Juge (§3.3.2).

3.3. FORMULATION DU PROBLÈME

3.3.4.0.1 Cas dégénéré. Lorsque le sous-ensemble admissible $M(e)$ défini par l’équation (3.5) est vide, le routeur retourne le statut “intent non routable”, noté $\perp$, plutôt que de violer un SLA. Le système renvoie alors une erreur explicite à l’opérateur, contenant le motif de non-admissibilité : dépassement de $L_{\max}(e)$, de $C_{\max}(e)$, ou des deux. Ce comportement diffère des cadres généralistes [32, 37] qui supposent implicitement $M(e) \neq \emptyset$. Dans un contexte réseau, la non-réponse est préférable à une réponse qui viole un SLA.

3.3.4.0.2 Résolution des égalités. Si plusieurs modèles atteignent le maximum dans l’équation (3.6), le routeur départage selon l’ordre lexicographique $(c_i, \ell_i(e))$ croissant : coût le plus faible en priorité, puis latence la plus faible à coût égal. Ce critère de départage est conforme à la projection Pareto-optimale du plan coût-qualité décrite par Hu et al. [47].

3.3.5 Hypothèses opérationnelles

La formulation précédente repose sur quatre hypothèses opérationnelles, que nous explicitons pour circonscrire le périmètre de validité de nos contributions et préciser les variables contrôlées au moment de l’évaluation.

3.3.5.0.1 H1 – Spécialisation par domaine. Pour chaque domaine fonctionnel $d \in \mathcal{D}$ (éq. (3.2)), nous supposons qu’un unique LLM spécialisé couvre le domaine, à l’exclusion de toute stratégie d’ensembling ou de cascade intra-domaine. Cette hypothèse isole la variable spécialisation de la variable architecture du modèle dans l’analyse expérimentale du chapitre 5. Elle est conforme aux protocoles comparés de la littérature de routage qualité/coût, où Ding et al. [32] et Dekoninck et al. [37] évaluent chacun un seul candidat par classe de complexité. La généralisation à un ensemble multi-modèle par domaine est laissée à des travaux ultérieurs.

3.3.5.0.2 H2 – Absence d’oracle humain au runtime. La mesure de qualité $q_i(e)$ qui calibre l’estimation $\hat{q}_i(e)$ (§3.3.2) est produite exclusivement par le LLM Juge local, sans intervention humaine au moment de l’inférence. Cette hypothèse répond à la contrainte temps réel des intents réseau : un opérateur ne peut tolérer un délai de validation humaine pour une reconfiguration de slice ou une réponse à un incident de sécurité. Elle suit la même logique que les évaluateurs automatisés adoptés

3.3. FORMULATION DU PROBLÈME

par RouterBench [47], en se distinguant par une exécution locale du Juge afin de maîtriser coût et confidentialité.

3.3.5.0.3 H3 – Charge système non observable en cloud. Les LLM cibles du pool M (éq. (3.3)) sont majoritairement accessibles via des API cloud propriétaires (typiquement OpenRouter), ce qui rend la charge GPU/CPU sous-jacente non observable depuis le routeur. Aucune décision de routage ne peut donc s'appuyer sur des indicateurs de saturation infrastructurelle, comme ce serait le cas dans une formulation de placement de charge classique. Cette hypothèse justifie le choix de fonder la décision sur des grandeurs observables côté client (latence mesurée $\ell_i(e)$, tarif unitaire c_i , estimation de qualité $\hat{q}_i(e)$) plutôt que sur des métriques infrastructurelles.

3.3.5.0.4 H4 – Indépendance des intents. Chaque intent $e \in E$ est traité de manière indépendante, sans mémoire des décisions de routage antérieures et sans contexte conversationnel partagé entre requêtes. Cette hypothèse simplifie l'évaluation comparative en éliminant les effets de cache et les biais de séquence, au prix d'ignorer des optimisations potentielles telles que la mise en cache des réponses pour intents récurrents. L'extension à un routage avec mémoire constitue une perspective discutée au chapitre 6.

3.3.6 Qualité agrégée (AIQ)

L'évaluation comparative menée au chapitre 5 repose sur quatre métriques quantitatives, couvrant les axes qualité, latence, coût et répartition. Nous fixons ici leur définition opérationnelle.

Soit $\mathcal{T} \subset E$ le jeu d'évaluation, composé d'intents réseau générés par LLM, couvrant les quatre domaines de l'équation (3.2) et annotés sur le domaine, la complexité et la criticité, puis revus par l'expertise réseau du projet. Sa construction est détaillée au chapitre 5 (§5.1.1). La qualité moyenne d'inférence (*Average Inference Quality*, AIQ), suivant la terminologie introduite par RouterBench [47], est définie par :

$$\text{AIQ} = \frac{1}{|\mathcal{T}|} \sum_{e \in \mathcal{T}} q_{R^*(e)}(e), \quad (3.7)$$

où $|\mathcal{T}|$ désigne le cardinal du jeu d'évaluation et $q_{R^*(e)}(e) \in [0, 1]$ la qualité mesurée a posteriori par le

3.3. FORMULATION DU PROBLÈME

LLM Juge sur la réponse du modèle sélectionné $R^*(e)$ (équ. (3.6)). Une AIQ élevée traduit une bonne adéquation du routeur à la difficulté des intents.

3.3.7 Latence bout en bout (P50, P99)

Soit $T(e)$ le temps total mesuré pour traiter un intent e , décomposé en :

$$T(e) = T_{\text{est}}(e) + T_{\text{rout}}(e) + T_{\text{inf}}(e) + T_{\text{juge}}(e), \quad (3.8)$$

où $T_{\text{est}}(e)$ est le temps d'estimation de complexité, $T_{\text{rout}}(e)$ le temps de décision du routeur, $T_{\text{inf}}(e)$ le temps d'inférence du modèle $R^*(e)$ et $T_{\text{juge}}(e)$ le temps d'évaluation par le LLM Juge.¹ Nous rapportons les percentiles $P_{50}(T)$ et $P_{99}(T)$ sur $\mathcal{T}$, conformément aux pratiques d'évaluation des systèmes en production [47]. La moyenne masquerait les pics de queue, qui déterminent le respect d'un SLA ; le P99 les expose. La contrainte opérationnelle est $P_{99}(T) \leq \max_{e \in \mathcal{T}} L_{\text{max}}(e)$.

3.3.8 Coût agrégé proxy

Pour chaque intent e , le coût d'inférence est le produit du tarif unitaire du modèle sélectionné $c_{R^*(e)}$ (en unités monétaires pour 1000 jetons, normalisé sur la grille tarifaire OpenRouter en vigueur au moment des expériences, cf. chapitre 5) par le nombre total $\text{tokens}(e)$ de jetons d'entrée et de sortie consommés. Le coût agrégé sur le jeu d'évaluation $\mathcal{T}$ correspond à la moyenne arithmétique des coûts par intent. Cette définition reprend la métrique de coût moyen de RouterBench [47] en l'adaptant au cas où le tarif dépend du modèle sélectionné par le routeur et non d'un modèle unique.

3.3.9 Distribution de routage

On note π_i la proportion d'intents du jeu d'évaluation $\mathcal{T}$ routés vers le modèle $m_i \in M$. Le vecteur de distribution $(\pi_1, \pi_2, \dots, \pi_k)$ permet de vérifier l'absence de "collapse" du routeur sur un sous-ensemble restreint du pool. On rapporte également la corrélation croisée entre la décision du routeur $R^*(e)$ et le domaine $d(e)$ de l'intent, qui diagnostique l'effectivité de la spécialisation par domaine posée en hypothèse H1 (§3.3.5.0.1).

¹La décomposition omet volontairement un terme de latence réseau $T_{\text{net}}(e)$. Le routeur étant co-localisé avec le contrôleur IBN sur un nœud de bordure, le transport intra-cluster reste de l'ordre de quelques millisecondes, négligeable devant $T_{\text{inf}}(e)$ qui dépasse typiquement 100 ms pour un LLM appelé en cloud.

3.3.10 Critère principal de comparaison

Les quatre métriques précédentes alimentent un plan coût / qualité dans lequel sont projetées les stratégies comparées (ALWAYS-HEAVY, ALWAYS-LIGHT, RANDOM, InferRouter-LLM). ALWAYS-HEAVY route chaque intent vers le modèle le plus capable, et le plus coûteux, du pool ; ALWAYS-LIGHT vers le moins coûteux ; RANDOM tire le modèle uniformément au hasard dans M . Ces trois stratégies ignorent le contenu de l'intent et encadrent le gain attribuable au routage informé. La métrique principale d'évaluation est la position de chaque stratégie sur la frontière de Pareto qualité/coût sous contrainte de latence, telle que définie par Hu et al. [47] : une stratégie est dite Pareto-dominante si elle réalise simultanément une AIQ supérieure et un coût agrégé inférieur à une stratégie de référence, tout en satisfaisant la contrainte $P_{99}(T) \leq \max_{e \in \mathcal{T}} L_{\max}(e)$. Cette projection synthétique sera l'objet central du chapitre 5.

Chapter 4

InferRouter-LLM

Contents

4.1	Vue d'ensemble de l'architecture	51
4.2	Contribution 1 : estimateur de complexité sémantique	53
4.3	Contribution 2 : LLM cibles spécialisés par domaine réseau	53
4.4	Contribution 3 : LLM Juge (RocketEval)	54
4.5	Auto-calibration des seuils	54

Chapitre en cours de consolidation

Ce chapitre décrit l'architecture telle qu'implémentée dans le prototype (campagne préliminaire de validation des risques) et telle qu'ajustée par les résultats du chapitre 5. Le système complet sera finalisé après la fiabilisation du juge et le benchmark comparatif.

Ce chapitre détaille l'architecture d'InferRouter-LLM et ses trois contributions. Il prolonge la formulation du chapitre 3 côté implémentation et anticipe l'évaluation du chapitre 5. Le système existe sous forme d'un prototype end-to-end, construit lors de cette campagne préliminaire pour valider au plus tôt les hypothèses qui portent tout le projet. Les choix exposés ici intègrent déjà ses enseignements.

4.1 Vue d'ensemble de l'architecture

InferRouter-LLM s'organise autour d'un routeur central qui reçoit un intent réseau textuel et choisit le LLM cible vers lequel déléguer l'inférence. La chaîne de traitement enchaîne quatre étapes. L'estimateur de complexité analyse l'énoncé et en déduit un niveau de difficulté. Le routeur croise ce niveau avec

4.1. VUE D'ENSEMBLE DE L'ARCHITECTURE

les attributs de coût et de latence des candidats, restreint le pool aux modèles admissibles (éq. (3.5)), puis sélectionne le modèle de plus haute qualité attendue $\hat{q}_i(e)$ (éq. (3.6)). Le LLM cible produit la réponse. Le LLM Juge local note cette réponse a posteriori, ce qui alimente la calibration des seuils hors ligne. La Figure 4.1 donne la vue d'ensemble de cette chaîne et de son déploiement.

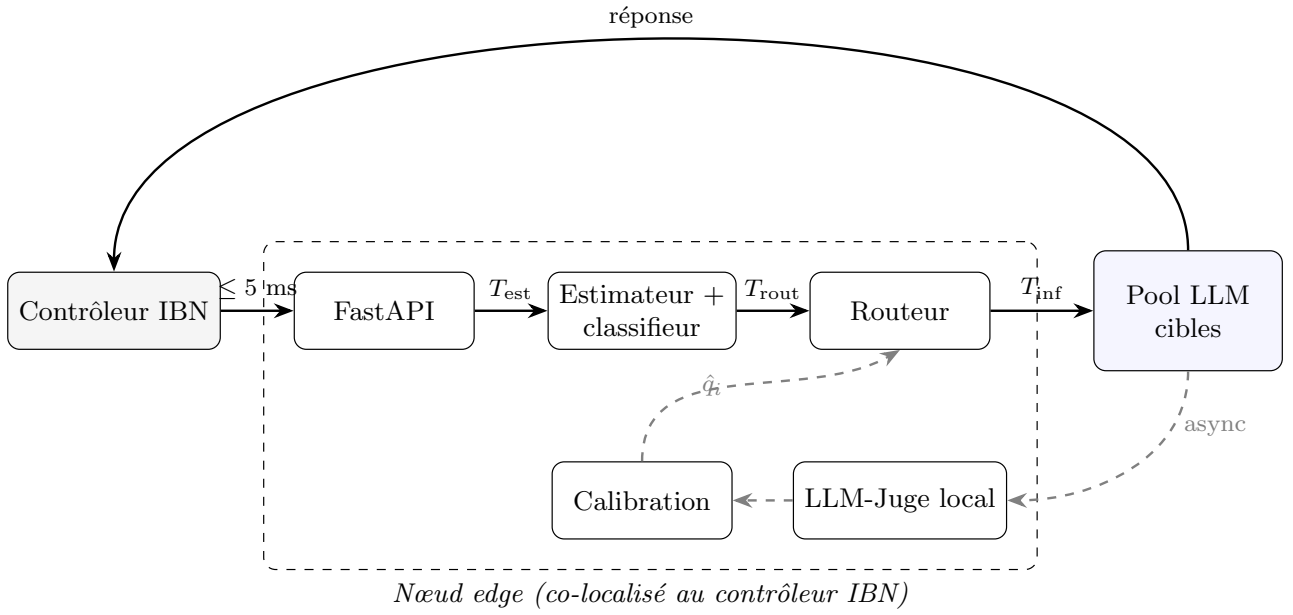


Figure 4.1: Architecture système d'InferRouter-LLM. Le chemin critique (traits pleins) enchaîne estimation, routage et inférence du modèle sélectionné $R^*(e)$, appelé en cloud. La co-localisation du routeur avec le contrôleur réseau rend le transport intra-cluster (≤ 5 ms) négligeable devant T_{inf} . Le LLM-Juge local évalue les réponses hors chemin critique (traits pointillés) et met à jour la table de calibration $\hat{q}_i$ utilisée par le routeur.

Le routeur n'exécute aucune inférence générative. Cette séparation isole la logique de décision des modèles sous-jacents et autorise l'ajout ou le retrait d'un candidat sans toucher au cœur du routage. Le routeur et le Juge sont co-localisés avec le contrôleur IBN sur un nœud de bordure, ce qui rend le transport réseau de l'intent négligeable devant l'inférence cloud des modèles cibles (§3.1.4).

Le prototype matérialise cette chaîne en modules Python découplés : un schéma d'intent, un client OpenRouter pour les modèles cibles, un wrapper du Juge local via Ollama, l'estimateur de complexité et la politique de routage. Le pool testé oppose un modèle léger, c'est-à-dire de coût unitaire c_i et de latence faibles mais de capacité moindre, à un modèle lourd, de grande capacité mais plus coûteux et plus lent. Cette configuration volontairement minimale isole le gain du routage.

4.2 Contribution 1 : estimateur de complexité sémantique

L'estimateur traduit les trois critères posés au chapitre 3 (§3.2) en attributs calculés sur l'énoncé : le nombre d'entités $n(e)$, la profondeur d'inférence $p(e)$ approximée par le nombre de contraintes (au sens fixé en §3.2.2), et le nombre de domaines croisés $|\delta(e)|$. Une tête `scikit-learn` (régression logistique ou forêt aléatoire) classe l'intent en trois niveaux, simple, medium ou complex, en moins de 15 ms.

Nous fondons l'estimateur sur ces attributs, et non sur la proximité d'embeddings bruts, pour deux raisons : ils implémentent directement les critères du chapitre 3, ce qui rend chaque décision explicable en termes d'entités, de contraintes et de domaines, et ils sont insensibles par construction à la verbosité de l'énoncé. Parmi eux, la profondeur d'inférence, captée par le nombre de contraintes, porte l'essentiel du pouvoir prédictif une fois la longueur neutralisée. L'évaluation nuance toutefois ce choix : sur les 252 intents, des embeddings de phrase séparent la complexité mieux que les attributs, et la combinaison des deux signaux fait encore mieux (§5.2.3). Les attributs restent le socle du prototype ; l'estimateur combiné est la piste du système final.

L'entraînement révèle un piège que nous documentons. Sur un jeu généré naturellement, la longueur de l'énoncé suffit à prédire la complexité, parce que les intents simples sont courts et les complexes longs. Ce confond produit une exactitude trompeuse. Mesuré à longueur contrôlée, le signal porté par les attributs se situe entre 65 et 73 %, bien au-dessus de la baseline de 33 %, mais loin du chiffre gonflé par la verbosité. Le chapitre 5 détaille cette mesure honnête.

4.3 Contribution 2 : LLM cibles spécialisés par domaine réseau

Le pool de modèles cibles mêle des LLM génériques, qui couvrent tous les domaines et servent de repli sur les intents multi-domaines, et des LLM spécialisés par domaine réseau (RAN, cœur, sécurité, slice). Le routeur arbitre selon la complexité estimée et les contraintes de l'intent. Un intent simple monodomaine peut être confié à un LLM spécialisé moins coûteux ; un intent multi-domaines exige un généraliste.

Le routeur sélectionne le candidat admissible de plus haute qualité attendue $\hat{q}_i(e)$, les égalités étant tranchées par le coût puis la latence (§3.3.4). Le prototype implante cette politique avec une estimation a priori de la qualité par candidat, calibrée ensuite par les mesures a posteriori $q_i(e)$ du Juge.

4.4. CONTRIBUTION 3 : LLM JUGE (ROCKETEVAL)

L'évaluation expose une condition de validité de cette contribution. Le gain du routage suppose qu'un modèle léger devienne compétitif sur les intents simples. Sur le couple testé (**llama-3.2-3b** contre **claude-sonnet-4.6**), le léger reste en dessous partout, y compris sur le simple, ce qui fait dégénérer le routage en stratégie ALWAYS-HEAVY (§3.3.10). Un léger plus fort (**gpt-4o-mini**) réduit l'écart sans le refermer sur cet échantillon. Le choix du pool devient donc un critère de conception à part entière, point développé en §5.4.

4.4 Contribution 3 : LLM Juge (RocketEval)

Le LLM Juge évalue la qualité d'une réponse par la méthode RocketEval [12], exécutée localement via Ollama. La méthode opère en deux temps. Un modèle fort génère hors ligne une checklist propre à chaque intent, soit une liste de critères vérifiables Y/N. Le juge local grade ensuite chaque item sur la réponse candidate, puis les notes sont agrégées en un score dans $[0, 1]$. La checklist par intent est le levier central : une checklist fixe identique pour tous les intents échoue, notamment sur l'aveuglement à l'hallucination (§5.3).

L'exécution locale tient à deux raisons. La latence d'évaluation reste maîtrisée et constante. Les intents réseau, potentiellement sensibles, ne quittent pas le périmètre de l'opérateur.

L'évaluation circonscrit précisément la portée de ce juge. En discrimination grossière, départager un bon candidat d'un mauvais, un juge local **gemma2:9b** doté de RocketEval est fiable. En discrimination fine, détecter une seule erreur injectée dans une réponse par ailleurs correcte, aucun juge local testé ne convient, et ni la taille ni le mode de notation ne corrigent ce point. Le routage ne demande que la discrimination grossière. Nous employons donc le juge comme signal de routage en comparaison relative entre candidats, et non comme oracle d'une qualité absolue fine. La contribution C3 tient pour cet usage, qui est celui qu'elle revendique. La fiabilisation du juge pour un usage plus fin est laissée à des travaux ultérieurs.

4.5 Auto-calibration des seuils

Le Juge sert aussi à calibrer les seuils du routeur hors ligne. La procédure mesure la qualité a posteriori $q_i(e)$ de chaque famille de modèles par niveau de complexité, puis ajuste l'estimation a priori $\hat{q}_i(e)$ qui guide la sélection (éq. (3.6)). Cette boucle relie le Juge et l'estimateur sans intervention humaine au

4.5. AUTO-CALIBRATION DES SEUILS

runtime, conformément à l'hypothèse H2 (§3.3.5).

La calibration s'appuie sur l'ordre relatif fourni par le juge, robuste, plutôt que sur ses valeurs absolues, qui se dégradent sur les intents complexes (§5.4). Tant que le juge reste fiable en comparaison grossière, la calibration garde son sens. Sa fiabilisation pour les cas fins reste un prérequis à une calibration plus précise, balisée dans le travail futur du chapitre 5.

Chapter 5

Évaluation

Contents

5.1	Protocole expérimental	57
5.1.1	Jeu de données d'intents	57
5.1.2	Modèles, juge et métriques	59
5.2	Estimateur de complexité sémantique	60
5.2.1	Un confond de longueur trompeur	60
5.2.2	Mesure honnête à longueur contrôlée	60
5.2.3	Embeddings de phrase : un signal réévalué	61
5.3	Fiabilité du LLM Juge	61
5.3.1	Checklist fixe contre RocketEval complet	61
5.3.2	Discrimination grossière contre discrimination fine	62
5.4	Prémisse du routage et calibration	63
5.4.1	Matrice de qualité par tier et complexité	63
5.4.2	Un léger plus compétitif	64
5.5	Synthèse et limites	64

Ce chapitre rend compte d'une campagne préliminaire de validation des risques, menée avant la construction du système complet. Le but n'était pas de produire un benchmark final, mais de tester les hypothèses dont dépend toute la chaîne : la fiabilité du LLM Juge, le pouvoir prédictif de la complexité sémantique, et la prémisse économique du routage. Plusieurs résultats sont négatifs. Nous les assumons comme tels, car ils ont chacun écarté une voie coûteuse avant qu'elle ne le devienne.

5.1 Protocole expérimental

5.1.1 Jeu de données d'intents

L'évaluation s'appuie sur un jeu de 252 intents réseau générés par LLM, réparti sur les quatre domaines fonctionnels de l'équation (3.2) (RAN, cœur, sécurité, slice) et trois niveaux de complexité (simple, medium, complex). La génération par LLM remplace le pipeline IBNs de Saha [48] : ce dernier produit des énoncés de flux Software Defined Networking (SDN) bas niveau, sans domaine telco ni annotation de complexité, inadaptés à notre architecture. Chaque intent porte six attributs, récapitulés dans la Table 5.1 avec leurs valeurs possibles et leur méthode d'attribution.

Table 5.1: Attributs d'un intent du jeu de données : valeurs possibles et méthode d'attribution.

Attribut	Valeurs possibles	Méthode d'attribution
id	slug court unique	produit à la génération
text	énoncé en langage naturel (anglais)	généré par LLM, dédoublé, revu
domain ($d(e)$)	ran, core, security, slice	fixé par la cellule de génération, revu
expected_complexity	simple, medium, complex	fixé par la cellule selon la grille de la Table 5.2, revu
criticality ($\kappa(e)$)	low, med, high	proposé par le générateur, revu
slice_type	embb, urllc, mmhc, null	proposé par le générateur, revu

L'annotation de complexité est reliée aux trois critères du chapitre 3 par la grille de la Table 5.2 : chaque niveau est défini par des bornes sur le nombre d'entités $n(e)$, la profondeur d'inférence $p(e)$ et le nombre de domaines croisés $|\delta(e)|$. Cette grille est imposée au générateur comme consigne, puis vérifiée en revue. Elle exige que le gradient de complexité soit porté par le contenu technique, jamais par la longueur de l'énoncé ; les attributs calculés de la §5.2.2 en sont des approximations mesurées ensuite automatiquement.

La génération procède par cellule de la matrice domaine $\times$ complexité : un appel au modèle générateur (claude-sonnet-4.6 via OpenRouter, température 0,7) produit un lot de 21 intents par couple, soit 12 cellules et 252 intents.

Le prompt de génération fixe le domaine et le niveau de complexité de la cellule, rappelle la grille de la Table 5.2, exige l'ancrage dans des scénarios 3GPP, O-RAN et ETSI ZSM avec des identifiants plausibles, et interdit les quasi-doublons. Trois intents de référence, tirés d'une amorce de 20 intents

5.1. PROTOCOLE EXPÉRIMENTAL

Table 5.2: Grille d’annotation de la complexité, exprimée sur les trois critères du chapitre 3. Au niveau complex, le croisement de domaines est majoritaire mais non systématique.

Niveau	$n(e)$	$p(e)$	$ \delta(e) $	Profil type
simple	1–2	1	1	lecture d’état ou requête directe, sans optimisation
medium	modéré	2	1	diagnostic guidé ou reconfiguration ciblée, une contrainte
complex	élevé	≥ 3	≥ 2	arbitrage multi-contraintes, plusieurs SLA simultanés

écrits à la main, servent d’exemples de style et de format.

Trois filtres automatiques s’appliquent à chaque lot. La validation de schéma écarte et journalise toute entrée malformée. Le domaine et la complexité sont forcés aux valeurs de la cellule, ce qui interdit toute dérive d’étiquette par le générateur. Une déduplication par texte normalisé élimine les doublons. [À FAIRE : Chiffrer le taux de rejet du filtrage (entrées malformées, doublons) à partir des journaux de génération.]

Chaque intent est ensuite revu manuellement par l’auteur, selon trois conventions arrêtées pendant la revue : la criticité mesure l’impact de l’action (une lecture de KPI ne dépasse pas med, hors lectures de sécurité) ; au niveau complex, le domaine désigne le point d’entrée de l’intent, pas l’unique champ mobilisé ; `slice_type` vaut null pour les intents multi-slices ou agnostiques de slice.

Cette revue a corrigé treize annotations (dix criticités sur-cotées, deux domaines, une complexité) sans écarter d’intent : le total reste 252, avec une matrice domaine $\times$ complexité quasi équilibrée (19 à 24 intents par cellule).

Un biais de génération est apparu dès les premiers entraînements : dans un jeu produit “naturellement” (variante v1), les intents simples sont courts et les intents complexes sont longs. La longueur seule devient alors un prédicteur quasi parfait, sans rapport avec le fond sémantique. Pour mesurer honnêtement le signal de complexité, nous avons généré deux variantes contrôlées. La variante v2 fixe une longueur quasi constante entre classes (simple 42, medium 41, complex 47 mots en moyenne). La variante v3 décorrèle la longueur de la complexité par tirage aléatoire d’une cible de longueur par intent, ce qui produit des plages chevauchantes (simple de 4 à 54 mots, complex de 9 à 126). La variante v1 reste le jeu naturel destiné au routeur et aux benchmarks ; v2 et v3 servent à mesurer la

5.1. PROTOCOLE EXPÉRIMENTAL

validité de l'estimateur.

5.1.2 Modèles, juge et métriques

Le pool de modèles cibles oppose un modèle léger à un modèle lourd, tous deux appelés via OpenRouter. Le léger initial est `llama-3.2-3b-instruct`, remplacé ensuite par `gpt-4o-mini` pour tester un candidat plus compétitif. Le lourd est `claude-sonnet-4.6`. Le LLM Juge s'exécute localement via Ollama, en `gemma2:2b` puis `gemma2:9b`, suivant la méthode RocketEval [12]. La Table 5.3 donne la taille et le tarif unitaire c_i de chaque modèle : près de deux ordres de grandeur séparent le léger initial du lourd. Les métriques sont celles du chapitre 3 : qualité agrégée (équ. (3.7)), latence P_{50} et P_{99} (équ. (3.8)), coût agrégé et distribution de routage.

Table 5.3: Modèles de la campagne : rôle, taille et tarif unitaire c_i en USD pour 1000 jetons (grille OpenRouter en vigueur au moment des expériences). n.c. : taille non communiquée par l'éditeur ; n.d. : tarif non consigné. Les juges locaux s'exécutent via Ollama, sans coût d'appel.

Modèle	Rôle	Taille (Mds)	c_i entrée	c_i sortie
<code>llama-3.2-3b-instruct</code>	cible légère initiale	3	0,000051	0,000335
<code>gpt-4o-mini</code>	cible légère (seconde)	n.c.	n.d.	n.d.
<code>claude-sonnet-4.6</code>	cible lourde ; génération du dataset et des check-lists	n.c.	0,003	0,015
<code>gemma2:2b</code>	juge local	2	exécution locale	
<code>gemma2:9b</code>	juge local	9	exécution locale	
Claude Opus	jugement de référence	n.c.	n.d.	n.d.

[À FAIRE : Compléter dans la Table 5.3 les tarifs OpenRouter de `gpt-4o-mini` et de Claude Opus tels qu'en vigueur lors des expériences.]

Le jugement de référence est produit en aveugle par un modèle fort (Claude Opus), sur des réponses anonymisées et mélangées. Cette calibration petit-juge contre juge-fort est une pratique standard [43, 40], mais un jeu validé par un expert humain resterait souhaitable. Les expériences sur l'estimateur utilisent une validation croisée à cinq plis ($k = 5$). Le coût des appels payants a été suivi tout au long : la campagne live a consommé environ 12 \$ sur OpenRouter, ce qui a borné la taille des échantillons et arrêté certains runs avant terme.

5.2 Estimateur de complexité sémantique

5.2.1 Un confond de longueur trompeur

Entraîné sur la variante naturelle v1, l'estimateur atteint 99,6 % d'exactitude en validation croisée. Le chiffre est un leurre. La seule variable `n_tokens`, c'est-à-dire le nombre de jetons de l'énoncé, atteint elle aussi 99,6 % : la complexité annotée est presque parfaitement corrélée à la longueur dans le jeu naturel. Nous écartons donc ce résultat. Le publier reviendrait à annoncer une estimation sémantique réussie alors que le modèle ne lit que la verbosité.

5.2.2 Mesure honnête à longueur contrôlée

L'extracteur d'attributs calcule six mesures par énoncé, par heuristiques lexicales sans modèle : `n_entities` (entités réseau distinctes, proxy de $n(e)$), `n_constraints` (marqueurs de contrainte et seuils SLA chiffrés, proxy de $p(e)$), `n_domains` (domaines fonctionnels évoqués, proxy de $|\delta(e)|$), `n_numbers` (valeurs numériques porteuses d'unité), `n_tokens` (mots) et `n_sentences` (phrases). Nous appelons attributs de fond les quatre premiers, qui ne portent aucune mesure de longueur.

Une fois la longueur neutralisée, le tableau change. Sur la variante v2, la longueur seule chute à 45,2 %, contre 99,6 % en v1 : le confond est cassé. Les attributs de fond atteignent 67,1 % ; un RandomForest sur l'ensemble des attributs monte à 73,0 %, pour une baseline majoritaire de 33,3 % (classes équilibrées). Le signal sémantique existe donc, au-delà du hasard. La Table 5.4 récapitule ces mesures, y compris pour les embeddings de phrase discutés en §5.2.3.

Table 5.4: Exactitude de l'estimateur de complexité par variante du jeu de données et par signal (validation croisée $k = 5$ sur 252 intents, régression logistique sauf mention RF, baseline majoritaire 33,3 %).

Signal	v1 (naturel)	v2 (long. contrôl.)	v3 (décorrélé)
Longueur seule	99,6 %	45,2 %	48,8 %
Attributs de fond	79,8 %	67,1 %	64,7 %
Tous attributs (RF)	99,2 %	73,0 %	73,0 %
Embeddings seuls	96,0 %	93,7 %	84,9 %
Attributs + embeddings	97,6 %	94,4 %	85,3 %

Parmi les attributs de fond, le nombre de contraintes (`n_constraints`), que nous utilisons comme proxy de la profondeur d'inférence $p(e)$ (§3.2.2), domine après contrôle de la longueur. Les seules

5.3. FIABILITÉ DU LLM JUGE

entités et domaines plafonnent autour de 50 % ; l'ajout des contraintes les fait passer à 63 %. La profondeur de raisonnement, et non le nombre d'entités, porte l'essentiel du pouvoir prédictif.

La variante v3 généralise mieux. Un estimateur entraîné sur v3 classe correctement quatre intents courts naturels sur cinq, contre deux sur cinq pour un estimateur entraîné sur v2. Nous retenons v3 comme jeu de référence de l'estimateur, parce que la décorrélation longueur-complexité le force à apprendre le fond plutôt que la verbosité. Le chiffre à retenir n'est pas le 99,6 % initial, mais les mesures obtenues sans confond : 65 à 73 % sur les attributs, jusqu'à 85 % en y joignant les embeddings.

5.2.3 Embeddings de phrase : un signal réévalué

Un premier test, mené sur les 20 intents de l'amorce manuelle (`all-MiniLM-L6-v2` suivi d'un k-NN en leave-one-out), prédisait la complexité à 35 %, soit le niveau de la baseline. Nous en avons conclu que l'espace sémantique encodait le sujet de l'intent, pas sa difficulté, et ce constat a orienté l'estimateur vers les attributs calculés.

Réévalué sur les 252 intents avec une régression logistique, ce verdict ne tient pas. Les embeddings seuls atteignent 93,7 % sur v2 et 84,9 % sur v3, au-dessus des attributs (73,0 %) ; la combinaison des deux signaux monte à 94,4 et 85,3 % (Table 5.4). La longueur n'explique pas ce résultat, puisqu'elle ne prédit que 48,8 % sur v3. Les mêmes embeddings prédisent aussi le domaine fonctionnel à 81,7 % (baseline 25,8 %). Le test initial concluait au hasard par manque d'échantillon et par faiblesse du classifieur, pas par absence de signal.

Ce constat ne renverse pas le choix du prototype, il le borne. Les attributs calculés restent lisibles en termes métier (entités, contraintes, domaines), implémentent directement les critères du chapitre 3 et sont insensibles à la verbosité par construction ; le modèle embarqué dans le routeur s'appuie sur eux. Pour le système final, l'estimateur combiné attributs + embeddings devient la référence à battre.

5.3 Fiabilité du LLM Juge

5.3.1 Checklist fixe contre RocketEval complet

Le premier juge note chaque réponse sur une checklist fixe de quatre critères binaires (oui/non), identiques pour tous les intents : correction technique au regard de l'intent, complétude de la réponse, adéquation au domaine réseau, absence de faits ou de valeurs inventés. Le score est la proportion

5.3. FIABILITÉ DU LLM JUGE

de critères satisfaits. Ce juge échoue. Sur 20 intents, son accord avec le jugement de référence est de 40 % en **gemma2:2b**, sous le seuil de 60 % que nous nous étions fixé. Passer à **gemma2:9b** ne suffit pas : l'accord monte à 50 %, toujours sous le seuil. Trois défauts reviennent : le juge récompense les hallucinations (note de 1,00 à une réponse qui invente un nombre d'User Equipment (UE) enregistrés), classe parfois la pire réponse au-dessus de la meilleure, et sature autour de 0,88 à 1,00 sur les intents complexes sans discriminer. Le levier n'est pas la taille du juge, c'est la méthode d'évaluation.

Nous remplaçons la checklist fixe par RocketEval complet [12] : une checklist générée par intent via un modèle fort (**claude-sonnet-4.6**), gradée ensuite par le même petit juge local. Un exemple de checklist générée figure en annexe A.2. L'accord passe à 90 % (18 sur 20) en **gemma2:2b**, et l'aveuglement à l'hallucination est corrigé : une réponse qui invente un chiffre tombe de 1,00 à 0,43, et l'ordre correct est rétabli. Un juge de deux milliards de paramètres doté d'une bonne checklist dépasse donc largement un juge de neuf milliards à checklist fixe. La méthode prime sur la taille, conformément à RocketEval.

5.3.2 Discrimination grossière contre discrimination fine

Le 90 % précédent est optimiste. Il oppose un modèle fort à un modèle faible, ce qui produit une référence sans variance : le lourd l'emporte sur les vingt intents, et un juge qui dirait toujours "lourd" obtiendrait 100 %. La métrique d'accord est donc dégénérée. Pour la corriger, nous construisons un test décisif : pour chaque intent, nous dégradons la bonne réponse en y injectant une seule erreur (chiffre faux, étape retirée, affirmation fausse), puis nous demandons au juge de classer l'originale et la dégradée. La vérité-terrain est connue et les deux textes sont quasi identiques.

Le juge échoue sur cette tâche fine. En **gemma2:2b**, la préférence correcte tombe à 35 %, avec 40 % d'égalités et 25 % d'inversions. Par type d'erreur, le chiffre faux est détecté à 57 %, l'affirmation fausse à 33 %, mais l'étape manquante à 14 % seulement, avec quatre inversions : retirer une étape critique fait souvent monter le score de la version dégradée, parce que le texte paraît plus propre. Grossir le juge n'aide pas : en **gemma2:9b**, la préférence correcte chute à 5 % avec 90 % d'égalités. La notation par paires, qui ne dépend pas de la checklist, descend à 0 % de préférence correcte et 100 % d'égalités : le juge répond sincèrement "égalité".

Deux régimes se distinguent nettement, résumés dans la Table 5.5. En discrimination grossière, un candidat nettement meilleur qu'un autre, le juge local est fiable : **gemma2:9b** avec RocketEval atteint

5.4. PRÉMISSE DU ROUTAGE ET CALIBRATION

100 %. En discrimination fine, détecter une erreur isolée, aucun juge local testé ne convient. Ni la méthode de notation ni la taille du modèle ne corrigent ce point. La cause est le discernement du juge léger lui-même.

Table 5.5: Accord du LLM Juge selon le régime de discrimination et la configuration. Grossier : bon candidat contre mauvais. Fin : réponse correcte contre variante à une erreur injectée.

Configuration	Grossier	Fin (préf. correcte)
Checklist fixe, <code>gemma2:2b</code>	40 %	n/a
Checklist fixe, <code>gemma2:9b</code>	50 %	n/a
RocketEval, <code>gemma2:2b</code>	90 %	35 %
RocketEval, <code>gemma2:9b</code>	100 %	5 %
RocketEval pairwise, <code>gemma2:9b</code>	n/a	0 %

La conséquence pour le système est directe. Le routage ne demande que la discrimination grossière : pour un intent donné, le LLM spécialisé est-il nettement meilleur que le modèle générique. Un juge local `gemma2:9b` avec RocketEval répond à cette question. Nous l’employons donc comme signal de routage en comparaison relative entre candidats, et non comme oracle de qualité absolue sur lequel poser un seuil. Cet usage est suffisant pour la contribution C3, et c’est ce qu’elle revendique.

5.4 Prémisse du routage et calibration

5.4.1 Matrice de qualité par tier et complexité

Le routage par qualité n’économise que si un modèle léger devient compétitif sur les intents simples. Nous mesurons donc la qualité réelle, notée par le juge, de chaque tier sur chaque niveau de complexité. Avec `llama-3.2-3b` face à `claude-sonnet-4.6`, le modèle léger est inadéquat partout, y compris sur le simple : qualité 0,11 contre 0,55 pour le lourd, soit un écart de 0,45. La Table 5.6 détaille la matrice. L’écart se maintient sur le medium (0,13 contre 0,50) et le complex (0,03 contre 0,22).

Table 5.6: Qualité moyenne notée par le juge, par tier de modèle et niveau de complexité (échantillon de calibration, 12 intents équilibrés).

Complexité	<code>llama-3.2-3b</code>	<code>gpt-4o-mini</code>	<code>claude-sonnet-4.6</code>
simple	0,11	0,24	0,55
medium	0,13	0,30	0,50
complex	0,03	0,06	0,22

5.5. SYNTHÈSE ET LIMITES

Le routage par “argmax q ” s’effondre alors en always-heavy : le léger ne suffit jamais, donc aucune économie de coût n’est possible sur ce couple. Ce résultat est instructif plutôt que décevant. Le gain du routage est conditionné par le choix du pool. Un modèle léger trop faible l’annule, ce qui fait du choix du pool un critère de conception à part entière.

5.4.2 Un léger plus compétitif

Nous testons un léger plus fort, **gpt-4o-mini**, en réutilisant le lourd et les checklists déjà calculés. Il double **llama** sur le simple (0,24 contre 0,11) et dépasse même le lourd sur un intent simple isolé (0,71 contre 0,57). En moyenne, il reste sous le lourd, avec un écart de 0,31 sur le simple contre 0,45 pour **llama**. La parité n’est pas démontrée sur cet échantillon. Un meilleur léger réduit l’écart, sans le refermer ici.

Une grande calibration de **gpt-4o-mini** face au lourd a été interrompue à 45 intents sur 72, crédits OpenRouter épuisés. Un défaut d’ordonnancement de l’échantillonneur a traité les classes complex puis medium avant simple : la ligne simple, pourtant la plus informative, n’a pas été recalculée à grande échelle. Le run reste donc partiel. **[À FAIRE : Relancer la calibration grande échelle avec entrelacement des classes (simple/medium/complex) et un budget OpenRouter suffisant, pour statuer sur la parité léger/lourd sur le simple.]**

Le facteur limitant de toute mesure de qualité est le juge local lui-même. Il est sévère et bruité sur le complexe : le lourd n’y obtient que 0,22, valeur anormalement basse pour **claude-sonnet**. Sur les intents complexes, le juge ne discrimine pas, ce qui rejoint le 35 % de discrimination fine de la §5.3.2. C’est l’ordre relatif, le lourd au-dessus du léger, qui est robuste, surtout sur le simple. Les valeurs absolues, elles, sont peu fiables.

5.5 Synthèse et limites

Deux acquis ressortent de cette campagne. D’abord, la complexité d’un intent porte un signal sémantique réel une fois la longueur neutralisée : 65 à 73 % sur les attributs calculés, jusqu’à 85 % avec les embeddings de phrase, pour une baseline de 33 %. Ensuite, un juge local **gemma2:9b** avec RocketEval suffit pour le routage, qui ne demande qu’une discrimination grossière, validée à 100 %. La contribution C3 tient pour cet usage.

Deux points ne sont pas établis. Le juge local ne sert pas d'oracle de qualité fine : il ne détecte pas une erreur isolée, et ni la taille ni la méthode de notation ne corrigent cette limite intrinsèque. Le gain de coût du routage n'est pas démontré non plus : sur les couples testés, aucun modèle léger n'atteint la qualité du lourd sur les intents simples, donc le routage par qualité dégénère en always-heavy. Ce constat est borné par la limite du juge et par la taille réduite des échantillons, non définitif.

Le travail futur découle de ces limites. Un juge nettement plus fort, via API ou grand modèle local, est un prérequis à toute mesure fine de qualité. Un modèle léger plus compétitif est nécessaire pour démontrer un gain de coût ; à défaut, le routage se reformule autour du domaine ou d'une dégradation assumée. Un jeu d'évaluation plus large libérerait les mesures de la contrainte de budget. Enfin, le benchmark complet des quatre stratégies (ALWAYS-HEAVY, ALWAYS-LIGHT, RANDOM, InferRouter-LLM) sur le plan coût/qualité (§3.3.10) reste à exécuter. [**À FAIRE** : Exécuter le benchmark comparatif des quatre stratégies sur le jeu v1 complet, une fois le juge fiabilisé et le pool recalibré, et reporter la projection sur la frontière de Pareto.]

Chapter 6

Conclusion

Contents

6.1	Synthèse	66
6.2	Apports scientifiques	66
6.3	Limites et perspectives	66

Chapitre en cours de rédaction

Ce chapitre sera finalisé après la phase expérimentale (conception du jeu de données d'intents et benchmarks). La présente version partielle est soumise pour validation de la nouvelle orientation du mémoire.

6.1 Synthèse

6.2 Apports scientifiques

6.3 Limites et perspectives

Bibliography

- [1] N. Maslej, L. Fattorini, R. Perrault, Y. Gil, V. Parli, N. Kariuki, E. Capstick, A. Reuel, E. Brynjolfsson, J. Etchemendy, K. Ligett, T. Lyons, J. Manyika, J. C. Niebles, Y. Shoham, R. Wald, T. Walsh, A. Hamrah, L. Santarlasci, J. B. Lotufo, A. Rome, A. Shi, and S. Oak, “Artificial Intelligence Index Report 2025,” AI Index Steering Committee, Institute for Human-Centered AI, Stanford Univ., Stanford, CA, USA, Tech. Rep., Apr. 2025, doi: 10.48550/arXiv.2504.07139. [Online]. Available: <https://hai.stanford.edu/ai-index/2025-ai-index-report>
- [2] A. Maatouk, N. Piovesan, F. Ayed, A. De Domenico, and M. Debbah, “Large language models for telecom: Forthcoming impact on the industry,” *IEEE Commun. Mag.*, vol. 63, no. 1, pp. 62–68, Jan. 2024, doi: 10.1109/MCOM.001.2300473.
- [3] M. El Rajab, L. Yang, and A. Shami, “Zero-touch networks: Towards next-generation network automation,” *Comput. Netw.*, vol. 243, p. 110294, Apr. 2024, doi: 10.1016/j.comnet.2024.110294.
- [4] E. Karakaya, O. Ercetin, H. Ozkan, M. Karaca, E. Dehghan Biyar, and A. Palaaios, “Online learning for autonomous management of intent-based 6G networks,” arXiv preprint arXiv:2407.17767, Jul. 2024, doi: 10.48550/arXiv.2407.17767.
- [5] A. Clemm, L. Ciavaglia, L. Z. Granville, and J. Tantsura, “Intent-based networking - concepts and definitions,” Internet Research Task Force (IRTF), RFC 9315, Oct. 2022, doi: 10.17487/RFC9315. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9315>
- [6] M. Falkner and J. Apostolopoulos, “Intent-based networking for the enterprise: A modern network architecture,” *Commun. ACM*, vol. 65, no. 11, pp. 108–117, Nov. 2022, doi: 10.1145/3538513.

BIBLIOGRAPHY

- [7] A. Leivadeas and M. Falkner, “A survey on intent-based networking,” *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 625–655, First Quarter 2023, doi: 10.1109/COMST.2022.3215919.
- [8] K. Mehmood, K. Kralevska, and D. Palma, “Intent-driven autonomous network and service management in future cellular networks: A structured literature review,” *Comput. Netw.*, vol. 220, p. 109477, Jan. 2023, doi: 10.1016/j.comnet.2022.109477.
- [9] Y. Huang, H. Du, X. Zhang, D. Niyato, J. Kang, Z. Xiong, S. Wang, and T. Huang, “Large language models for networking: Applications, enabling techniques, and challenges,” *IEEE Netw.*, vol. 38, no. 5, pp. 235–242, Sep. 2024, doi: 10.1109/MNET.2024.3435752.
- [10] L. Chen, M. Zaharia, and J. Zou, “FrugalGPT: How to use large language models while reducing cost and improving performance,” *Trans. Mach. Learn. Res. (TMLR)*, May 2024, doi: 10.48550/arXiv.2305.05176.
- [11] J. W.-K. Hong, “A comprehensive survey on LLM-based network management and operations,” *Int. J. Netw. Manag.*, vol. 35, no. 6, p. e70029, Nov. 2025, doi: 10.1002/nem.70029.
- [12] T. Wei, W. Wen, R. Qiao, X. Sun, and J. Ma, “RocketEval: Efficient automated LLM evaluation via grading checklist,” in *Proc. 13th Int. Conf. Learn. Representations (ICLR)*, Singapore, Apr. 2025, doi: 10.48550/arXiv.2503.05142.
- [13] L. Pang, C. Yang, D. Chen, Y. Song, and M. Guizani, “A survey on intent-driven networks,” *IEEE Access*, vol. 8, pp. 22 862–22 873, Jan. 2020, doi: 10.1109/ACCESS.2020.2969208.
- [14] *Zero-touch network and Service Management (ZSM); Intent-driven autonomous networks; Generic aspects*, ETSI GR ZSM 011, Rev. 1.1.1, Feb. 2023. [Online]. Available: https://www.etsi.org/deliver/etsi_gr/ZSM/001_099/011/01.01.01_60/gr_ZSM011v010101p.pdf
- [15] E. Zeydan and Y. Turk, “Recent advances in intent-based networking: A survey,” *Proc. IEEE 91st Veh. Technol. Conf. (VTC2020-Spring)*, pp. 1–5, May 2020, doi: 10.1109/VTC2020-Spring48590.2020.9128422.
- [16] A. S. Jacobs, R. J. Pfitscher, R. H. Ribeiro, R. A. Ferreira, L. Z. Granville, W. Willinger, and S. G. Rao, “Hey, Lumi! Using natural language for intent-based network management,” in *Proc.*

BIBLIOGRAPHY

- 2021 *USENIX Annu. Tech. Conf. (USENIX ATC '21)*. USENIX Association, Jul. 2021, pp. 625–639. [Online]. Available: <https://www.usenix.org/conference/atc21/presentation/jacobs>
- [17] C. Liu, X. Xie, X. Zhang, and Y. Cui, “Large language models for networking: Workflow, advances and challenges,” arXiv preprint arXiv:2404.12901, Apr. 2024, doi: 10.48550/arXiv.2404.12901.
- [18] H. Zhou, C. Hu, Y. Yuan, Y. Cui, Y. Jin, C. Chen, H. Wu, D. Yuan, L. Jiang, D. Wu, X. Liu, C. Zhang, X. Wang, and J. Liu, “Large language model (LLM) for telecommunications: A comprehensive survey on principles, key techniques, and opportunities,” arXiv preprint arXiv:2405.10825, May 2024, doi: 10.48550/arXiv.2405.10825.
- [19] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang, “NetLLM: Adapting large language models for networking,” in *Proc. ACM SIGCOMM 2024 Conf.*, Sydney, Australia, Aug. 2024, pp. 661–678, doi: 10.1145/3651890.3672268.
- [20] D. M. Manias, A. Chouman, and A. Shami, “Semantic routing for enhanced performance of LLM-assisted intent-based 5G core network management and orchestration,” in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Cape Town, South Africa, Dec. 2024, pp. 1–6, doi: 10.1109/GLOBECOM52923.2024.10901594.
- [21] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, “NetConfEval: Can LLMs facilitate network configuration?” *Proc. ACM Netw.*, vol. 2, no. CoNEXT2, p. Art. no. 7, Jun. 2024, doi: 10.1145/3656296.
- [22] R. Mondal, A. Tang, R. Beckett, T. Millstein, and G. Varghese, “What do LLMs need to synthesize correct router configurations?” in *Proc. 22nd ACM Workshop Hot Topics Networks (HotNets '23)*, Cambridge, MA, USA, Nov. 2023, pp. 189–195, doi: 10.1145/3626111.3628194.
- [23] B. Ifland, E. Duani, R. Krief, M. Ohana, A. Zilberman, A. Murillo, O. Manor, O. Lavi, H. Kenji, A. Shabtai, Y. Elovici, and R. Puzis, “GeNet: A multimodal LLM-based co-pilot for network topology and configuration,” arXiv preprint arXiv:2407.08249, Jul. 2024, doi: 10.48550/arXiv.2407.08249.
- [24] F. Ayed, A. Maatouk, N. Piovesan, A. De Domenico, M. Debbah, and Z.-Q. Luo, “Hermes: A large language model framework on the journey to autonomous networks,” arXiv preprint arXiv:2411.06490, Nov. 2024, doi: 10.48550/arXiv.2411.06490.

BIBLIOGRAPHY

- [25] D. M. Manias, A. Chouman, and A. Shami, “Towards intent-based network management: Large language models for intent extraction in 5G core networks,” in *Proc. 2024 20th Int. Conf. Design Reliable Commun. Netw. (DRCN)*, Montreal, Canada, May 2024, pp. 1–6, doi: 10.1109/DRCN60692.2024.10539183.
- [26] A. Mekrache and A. Ksentini, “LLM-enabled intent-driven service configuration for next generation networks,” in *Proc. 2024 IEEE 10th Int. Conf. Netw. Softwarization (NetSoft)*, Saint Louis, MO, USA, Jun. 2024, pp. 253–257, doi: 10.1109/NetSoft60951.2024.10588881.
- [27] Y. Wei, X. Xie, T. Hu, Y. Zuo, X. Chen, K. Chi, and Y. Cui, “INTA: Intent-based translation for network configuration with LLM agents,” in *Proc. IEEE 33rd Int. Conf. Netw. Protocols (ICNP)*, Oct. 2025, doi: 10.48550/arXiv.2501.08760.
- [28] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, “RouteLLM: Learning to route LLMs from preference data,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, Singapore, Apr. 2025, doi: 10.48550/arXiv.2406.18665.
- [29] K. Lu, H. Yuan, R. Lin, J. Lin, Z. Yuan, C. Zhou, and J. Zhou, “Routing to the expert: Efficient reward-guided ensemble of large language models,” in *Proc. 2024 Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol. (NAACL-HLT)*, Mexico City, Mexico, Jun. 2024, pp. 1964–1974, doi: 10.18653/v1/2024.naacl-long.109.
- [30] S. Chen, W. Jiang, B. Lin, J. T. Kwok, and Y. Zhang, “RouterDC: Query-based router by dual contrastive learning for assembling large language models,” in *Proc. 38th Conf. Neural Inf. Process. Syst. (NeurIPS)*, Vancouver, Canada, Dec. 2024, doi: 10.48550/arXiv.2409.19886.
- [31] S. N. Hari and M. Thomson, “Tryage: Real-time, intelligent routing of user prompts to large language models,” arXiv preprint arXiv:2308.11601, Aug. 2023, doi: 10.48550/arXiv.2308.11601.
- [32] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Ruhle, L. V. S. Lakshmanan, and A. H. Awadallah, “Hybrid LLM: Cost-efficient and quality-aware query routing,” in *Proc. Int. Conf. Learning Representations (ICLR)*, Vienna, Austria, May 2024, doi: 10.48550/arXiv.2404.14618.
- [33] A. Madaan, P. Aggarwal, A. Anand, S. P. Potharaju, S. Mishra, P. Zhou, A. Gupta, D. Rajagopal, K. Kappaganthu, Y. Yang, S. Upadhyay, Mausam, and M. Faruqui, “AutoMix: Automatically

BIBLIOGRAPHY

- mixing language models,” in *Proc. 38th Conf. Neural Inf. Process. Syst. (NeurIPS)*, Vancouver, Canada, Dec. 2024, doi: 10.48550/arXiv.2310.12963.
- [34] D. Jiang, X. Ren, and B. Y. Lin, “LLM-Blender: Ensembling large language models with pairwise ranking and generative fusion,” in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics (ACL), Vol. 1 (Long Papers)*, Toronto, Canada, Jul. 2023, pp. 14 165–14 178, doi: 10.18653/v1/2023.acl-long.792.
- [35] Z. Zhao, S. Jin, and Z. M. Mao, “Eagle: Efficient training-free router for multi-LLM inference,” in *Proc. 38th Conf. Neural Inf. Process. Syst. (NeurIPS) Workshops*, Vancouver, Canada, Dec. 2024, doi: 10.48550/arXiv.2409.15518.
- [36] T. Shnitzer, A. Ou, M. Silva, K. Soule, Y. Sun, J. Solomon, N. Thompson, and M. Yurochkin, “Large language model routing with benchmark datasets,” in *Proc. Conf. Lang. Model. (COLM)*, Philadelphia, PA, USA, Oct. 2024, doi: 10.48550/arXiv.2309.15789.
- [37] J. Dekoninck, M. Baader, and M. Vechev, “A unified approach to routing and cascading for LLMs,” arXiv preprint arXiv:2410.10347, Oct. 2024, doi: 10.48550/arXiv.2410.10347.
- [38] J. Zhang, R. Krishna, A. H. Awadallah, and C. Wang, “EcoAssistant: Using LLM assistant more affordably and accurately,” in *Proc. COLM Workshop / arXiv*, Oct. 2023, doi: 10.48550/arXiv.2310.03046.
- [39] W. Song, Z. Huang, C. Cheng, W. Gao, B. Xu, G. Zhao, F. Wang, and R. Wu, “IRT-Router: Effective and interpretable multi-LLM routing via Item Response Theory,” in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Vienna, Austria, 2025, doi: 10.48550/arXiv.2506.01048.
- [40] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu, S. Wang, K. Zhang, Y. Wang, W. Gao, L. Ni, and J. Guo, “A survey on LLM-as-a-judge,” arXiv preprint arXiv:2411.15594, Nov. 2024, doi: 10.48550/arXiv.2411.15594.
- [41] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating text generation with BERT,” in *Proc. 8th Int. Conf. Learn. Representations (ICLR)*, Addis Ababa, Ethiopia, Apr. 2020, doi: 10.48550/arXiv.1904.09675. [Online]. Available: <https://openreview.net/forum?id=SkeHuCVFDr>

BIBLIOGRAPHY

- [42] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, “G-Eval: NLG evaluation using GPT-4 with better human alignment,” in *Proc. 2023 Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Singapore, Dec. 2023, pp. 2511–2522, doi: 10.18653/v1/2023.emnlp-main.153.
- [43] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Proc. 37th Conf. Neural Inf. Process. Syst. (NeurIPS) Datasets and Benchmarks Track*, New Orleans, LA, USA, Dec. 2023, doi: 10.48550/arXiv.2306.05685.
- [44] L. Zhu, X. Wang, and X. Wang, “JudgeLM: Fine-tuned large language models are scalable judges,” in *Proc. 13th Int. Conf. Learn. Representations (ICLR), Spotlight*, Singapore, Apr. 2025, doi: 10.48550/arXiv.2310.17631.
- [45] Y. Wang, Z. Yu, Z. Zeng, L. Yang, C. Wang, H. Chen, C. Jiang, R. Xie, J. Wang, X. Xie, W. Ye, S. Zhang, and Y. Zhang, “PandaLM: An automatic evaluation benchmark for LLM instruction tuning optimization,” in *Proc. 12th Int. Conf. Learn. Representations (ICLR)*, Vienna, Austria, May 2024, doi: 10.48550/arXiv.2306.05087.
- [46] 3GPP, “Service requirements for the 5G system; Stage 1,” 3rd Generation Partnership Project (3GPP), Tech. Spec. (TS) 22.261, Sep. 2023, rel. 18, V18.6.0. [Online]. Available: https://www.3gpp.org/ftp/Specs/archive/22_series/22.261
- [47] Q. J. Hu, J. Bieker, X. Li, N. Jiang, B. Keigwin, G. Ranganath, K. Keutzer, and S. K. Upadhyay, “RouterBench: A benchmark for multi-LLM routing system,” in *Proc. Int. Conf. Mach. Learn. (ICML) Workshops*, Vienna, Austria, Jul. 2024, doi: 10.48550/arXiv.2403.12031.
- [48] B. Saha, “IBNs: Intent-Based Networking dataset generator,” <https://github.com/barun-saha/IBNs>, 2024, open-source pipeline for generating natural-language network intents.

BIBLIOGRAPHY

Appendix A

Annexes techniques

Contents

A.1	Configuration du système	70
A.2	Exemples d'intents et réponses	70

A.1 Configuration du système

[À FAIRE : Rédiger la configuration du système (versions des modèles, paramètres Ollama et Open-Router, machine d'exécution).]

A.2 Exemples d'intents et réponses

A.2.1 Un intent annoté du jeu de données

L'intent suivant, issu de l'amorce écrite à la main, illustre le schéma d'annotation décrit en §5.1.1 (attributs de la table 5.1) : une lecture d'état directe, un seul domaine, une à deux entités, donc un niveau simple selon la grille de la table 5.2.

```
id: ran-read-throughput
text: "What is the current downlink throughput on cell gNB-042?"
domain: ran
expected_complexity: simple
criticality: low
```

`slice_type: null`

A.2.2 Checklist RocketEval générée pour cet intent

La checklist ci-dessous a été générée par `claude-sonnet-4.6` pour l'intent précédent, selon la méthode RocketEval décrite en §5.3.1. Elle compte sept critères binaires, propres à cet intent ; le juge local répond oui ou non à chacun, et le score est la proportion de critères satisfaits. Les critères sont reproduits tels quels (la chaîne d'évaluation opère en anglais).

1. "Does the response avoid fabricating or inventing a specific numeric downlink throughput value for gNB-042 that it cannot know without live network access?"
2. "Does the response explain at least one concrete method (e.g., CLI command, SNMP OID, O1/NETCONF query, vendor-specific NMS tool, or KPI counter) that an operator can use to retrieve the current downlink throughput for gNB-042?"
3. "Does the response reference technically correct RAN-domain metrics or counters relevant to gNB-042 downlink throughput (e.g., DRB.UEThpDl, PDCP layer throughput, PRB utilization, or equivalent 5G NR KPIs)?"
4. "Does the response clearly acknowledge that it has no live access to the network and therefore cannot provide the current real-time value?"
5. "Does the response avoid presenting any placeholder or example throughput figure as if it were the actual current value for gNB-042?"
6. "Does the response correctly distinguish between downlink-specific metrics and uplink or aggregate metrics when describing how to obtain the data?"
7. "Does the response provide actionable guidance that is specific enough for a network operator to follow without requiring significant additional research?"

Les deux premiers critères relèvent de la couverture obligatoire imposée au générateur de checklists (détection de la fabrication de données, explication du moyen d'obtenir la donnée) ; le troisième porte la correction technique propre au domaine RAN. C'est ce type d'item vérifiable qui corrige l'aveuglement à l'hallucination constaté avec la checklist fixe (§5.3.1).

